AUTOM-07: CI/CD Integration

> **Series:** AUTOM — Dynatrace Automation | **Notebook:** 7 of 9 | **Created:** January 2026 | **Last Updated:** 11/11/2026

CI/CD integration brings software development practices to Dynatrace configuration management. By storing configs in Git and deploying via pipelines, teams gain version control, review processes, and automated deployments.

Table of Contents

1. Introduction
2. GitOps Fundamentals
3. GitHub Actions
   - Terraform with Combined Auth
   - Vault Integration
   - Policy-as-Code Gates
   - Drift Detection
   - Reusable Workflows
4. GitLab CI/CD
5. Bitbucket Pipelines
   - Pipeline — Monaco Deploy
   - Pipeline — Terraform with Combined Auth
   - Terraform Bitbucket Provider
   - Combined Workspace — Bitbucket and Dynatrace in One Apply
   - Variables and Secrets
   - OIDC Note
6. Atlassian Bamboo (with Bitbucket + S3)
   - Stack Assumed
   - Plan Specs YAML — Terraform with Combined Auth
   - PR Workflow with Plan Branches
   - Decoupled S3 State Files
   - Bamboo Variables and Credentials
   - Manual Stages for Production Gating
7. Azure DevOps Pipelines
   - Stack Assumed
   - Pipeline YAML — Terraform with Combined Auth
   - Environments and Approval Gates (configured in UI, not YAML)
   - Variable Groups for Dynatrace Credentials
   - PR-Driven Plan-Only Workflow
8. ArgoCD Integration
9. FluxCD Integration
10. Dynatrace Operator GitOps Patterns
11. Ephemeral Platform Token Minting Pattern
    - Conceptual Model — Four-Layer Trust Chain
    - Prerequisites — OAuth Client + Service Users + IAM Policies
    - Worked Recipe — Pre-Mint Sweep, Mint, Use, Delete

    - Capacity Planning Under the Per-User Cap
    - When This Pattern Is Justified
12. Best Practices
13. Governance Architecture
14. Next Steps

Prerequisites

Before starting this notebook, ensure you have:

Requirement	Description
CI/CD Platform	GitHub Actions, GitLab CI, Bitbucket Pipelines, Bamboo, or Azure DevOps
Monaco or Terraform	One of the config-as-code tools
Git Repository	For storing configurations
Authentication	API Token + Platform Token for full coverage (see AUTOM-04)
HashiCorp Vault	*Optional* — for runtime credential retrieval instead of static CI/CD secrets
OPA / Conftest	*Optional* — for policy-as-code gates in pipelines

Token Types Reminder

As of Dynatrace Terraform provider **v1.88.0**, synthetic monitors and classic SLOs (`dynatrace_slo` , `dynatrace_slo_v2`) require a classic **API Token** (`dt0c01`); the platform SLO resource `dynatrace_platform_slo` uses OAuth. For full resource coverage in pipelines, use **Platform Token + API Token** together. See AUTOM-04: Terraform Provider for details.

Learning Objectives

By the end of this notebook, you will:

- Understand GitOps principles for Dynatrace
- Know how to set up CI/CD pipelines for config deployment
- Be able to implement pull request workflows with plan-as-PR-comment
- Handle multi-environment deployments with combined auth
- Integrate HashiCorp Vault for runtime credential retrieval
- Implement policy-as-code gates with OPA/Conftest and Sentinel
- Set up automated drift detection workflows
- Create reusable GitHub Actions workflows for multi-team organizations

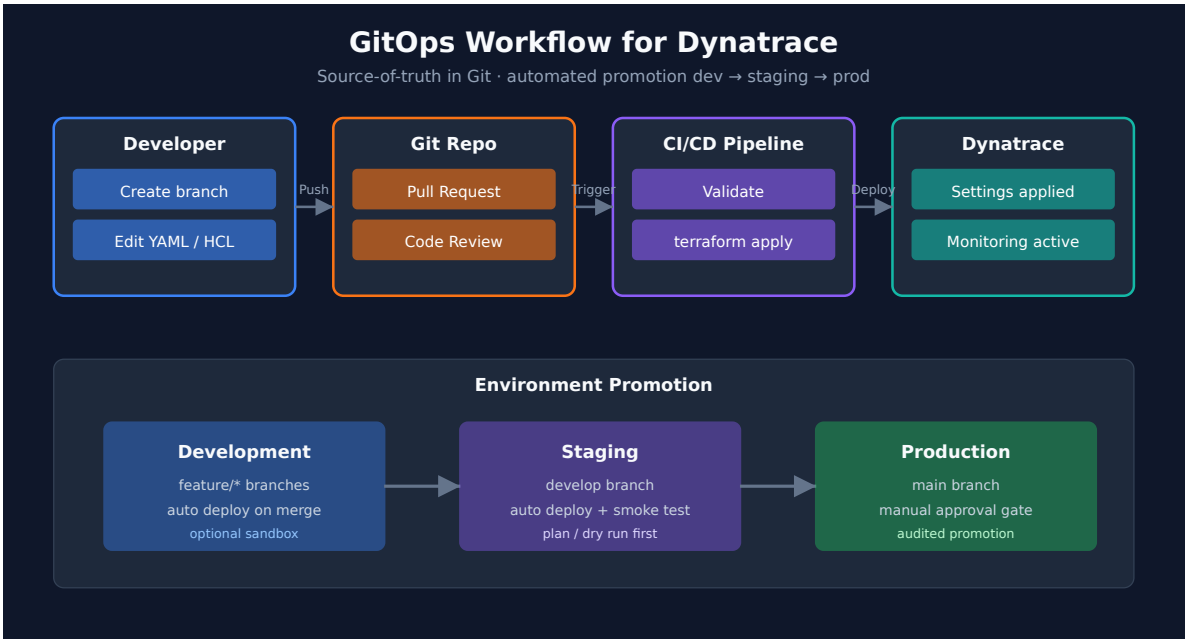
---

1. Introduction

Why CI/CD for Dynatrace?

Benefit	Description
Version Control	All changes tracked in Git history
Code Review	Pull requests for config changes
Automation	No manual deployments
Rollback	Revert to any previous state
Audit Trail	Who changed what, when

GitOps Workflow



2. GitOps Fundamentals

Repository Structure

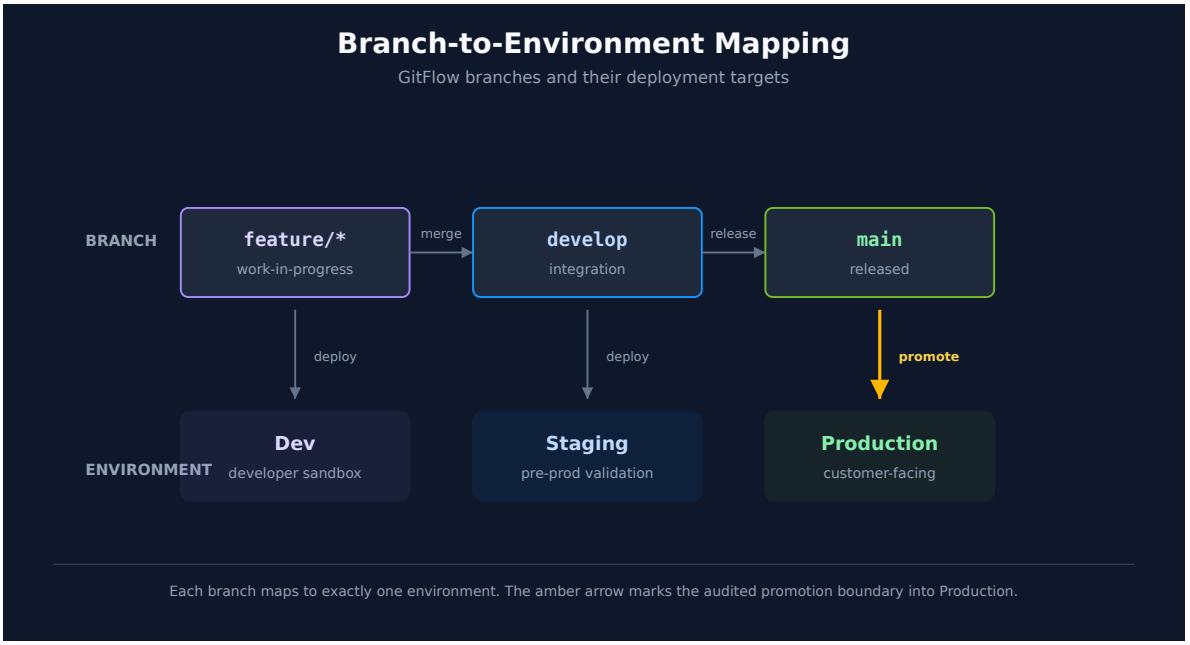
```
dynatrace-config/
├── .github/
│   └── workflows/
│       ├── validate.yml
│       └── deploy.yml
├── manifest.yaml          # Monaco manifest
├── environments/
│   ├── dev.yaml
│   ├── staging.yaml
│   └── production.yaml
├── projects/
│   ├── alerting/
│   ├── management-zones/
│   └── synthetic/
└── README.md
```

Branch Strategy

Branch	Purpose	Deploys To
`main`	Production configs	Production
`develop`	Integration branch	Staging
`feature/*`	New features	Dev (optional)

Environment Promotion

The table and diagram show the GitFlow variant. The GitHub Actions, GitLab and Bitbucket pipelines in §3–§5 use a simpler trunk-based variant: a merge to `main` deploys to staging, then production behind a manual approval. To get the GitFlow mapping, add `develop` to the deploy trigger and run the production job only when `github.ref == 'refs/heads/main'`.



3. GitHub Actions

Validation Workflow

.github/workflows/validate.yml:

```
name: Validate Dynatrace Config

on:
  pull_request:
    branches: [main, develop]
    paths:
      - 'projects/**'
      - 'manifest.yaml'

jobs:
  validate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7

      - name: Install Monaco
        run: |
          curl -L https://github.com/Dynatrace/dynatrace-configuration-as-
code/releases/latest/download/monaco-linux-amd64 -o monaco
          chmod +x monaco
          sudo mv monaco /usr/local/bin/

      # Monaco has no "validate" command — the dry run below is the validation step
      - name: Dry Run Deploy
        env:
          DT_DEV_URL: ${{ secrets.DT_DEV_URL }}
          DT_DEV_TOKEN: ${{ secrets.DT_DEV_TOKEN }}
        run: |
          monaco deploy manifest.yaml --environment development --dry-run
```

Deployment Workflow

.github/workflows/deploy.yml:

```
name: Deploy Dynatrace Config

on:
  push:
    branches: [main]
    paths:
      - 'projects/**'
      - 'manifest.yaml'

jobs:
  deploy-staging:
    runs-on: ubuntu-latest
    environment: staging
    steps:
      - uses: actions/checkout@v7

      - name: Install Monaco
        run: |
          curl -L https://github.com/Dynatrace/dynatrace-configuration-as-
code/releases/latest/download/monaco-linux-amd64 -o monaco
          chmod +x monaco
          sudo mv monaco /usr/local/bin/

      - name: Deploy to Staging
        env:
          DT_STAGING_URL: ${{ secrets.DT_STAGING_URL }}
          DT_STAGING_TOKEN: ${{ secrets.DT_STAGING_TOKEN }}
        run: |
          monaco deploy manifest.yaml --environment staging

  deploy-production:
    needs: deploy-staging
    runs-on: ubuntu-latest
    environment: production
    steps:
      - uses: actions/checkout@v7

      - name: Install Monaco
        run: |
          curl -L https://github.com/Dynatrace/dynatrace-configuration-as-
code/releases/latest/download/monaco-linux-amd64 -o monaco
          chmod +x monaco
          sudo mv monaco /usr/local/bin/

      - name: Deploy to Production
        env:
          DT_PROD_URL: ${{ secrets.DT_PROD_URL }}
          DT_PROD_TOKEN: ${{ secrets.DT_PROD_TOKEN }}
        run: |
          monaco deploy manifest.yaml --environment production
```

> **Optional — iterating locally.** These workflows run on GitHub-hosted runners, but you can also run them on your machine with `act` (Docker required) to catch YAML or step-ordering mistakes without a push cycle. It's a convenience for the *pipeline wiring*, not a check on the *config change* — `act` emulates the runner, it never talks to your tenant, so the load-bearing guardrail stays `monaco deploy --dry-run` (above) or `terraform plan`. One caution worth knowing: `act` does not honor GitHub Environment protection, so an `environment: production` approval gate like the one in the deploy workflow above is silently bypassed — don't point it at a production-deploy job.

Terraform Workflow with Combined Auth

For Terraform-based deployments, use **Platform Token + API Token** together for full resource coverage. The provider automatically routes each resource to the correct token.

```
name: Terraform Dynatrace

on:
  pull_request:
    branches: [main]
  push:
    branches: [main]

jobs:
  terraform:
    runs-on: ubuntu-latest
    permissions:
      contents: read
      pull-requests: write
    steps:
      - uses: actions/checkout@v7

      - name: Setup Terraform
        uses: hashicorp/setup-terraform@v4
        with:
          terraform_version: latest

      - name: Terraform Init
        run: terraform init

      - name: Terraform Plan
        id: plan
        if: github.event_name == 'pull_request'
        run: terraform plan -no-color -out=tfplan
        env:
          DYNATRACE_ENV_URL: ${{ secrets.DT_ENV_URL }}
          DYNATRACE_PLATFORM_TOKEN: ${{ secrets.DT_PLATFORM_TOKEN }}
          DYNATRACE_API_TOKEN: ${{ secrets.DT_API_TOKEN }}
          DYNATRACE_HTTP_OAUTH_PREFERENCE: "true"

      - name: Post Plan to PR
        if: github.event_name == 'pull_request'
        uses: borchero/terraform-plan-comment@v3
        with:
          token: ${{ github.token }}
          planfile: tfplan

      - name: Terraform Apply
        if: github.ref == 'refs/heads/main' &amp;&amp; github.event_name == 'push'
        run: terraform apply -auto-approve
        env:
          DYNATRACE_ENV_URL: ${{ secrets.DT_ENV_URL }}
          DYNATRACE_PLATFORM_TOKEN: ${{ secrets.DT_PLATFORM_TOKEN }}
          DYNATRACE_API_TOKEN: ${{ secrets.DT_API_TOKEN }}
          DYNATRACE_HTTP_OAUTH_PREFERENCE: "true"
```

Secret	Token Type	Covers
`DT_PLATFORM_TOKEN`	`dt0s16.xxxx`	Settings 2.0 + Gen3 Platform (workflows, documents, segments)
`DT_API_TOKEN`	`dt0c01.xxxx`	Synthetic monitors, classic SLOs (`dynatrace_slo` , `dynatrace_slo_v2`) — v1.88.0 requirement

Secret	Token Type	Covers
`DT_ENV_URL`	—	Tenant URL

> **Why combined auth?** A single API Token cannot manage Gen3 resources. A single Platform Token cannot manage synthetics or classic SLOs (OAuth removed for them in v1.88.0). Using both together gives the pipeline full coverage.
>
> **Action versions (checked 10/02/2026).** GitHub's runners moved to Node 24 by default on June 16, 2026 and, per the Node 20 deprecation notice (GitHub changelog), *"remove Node20 on September 23rd, 2026"*. The majors used in this notebook — `checkout@v7`, `setup-terraform@v4`, `terraform-plan-comment@v3`, `vault-action@v4`, `github-script@v9`, `slack-github-action@v4` — all declare `node24` in their `action.yml`; the older majors (`terraform-plan-comment@v2`, `vault-action@v3`, `github-script@v7`, `slack-github-action@v1`) declare `node20`.

Vault Integration for Runtime Credentials

For enterprise environments with **short-lived tokens** (e.g., 24-hour expiration), retrieve credentials at runtime from **HashiCorp Vault** instead of storing them as static CI/CD secrets.

```
name: Terraform with Vault Credentials

on:
  pull_request:
    branches: [main]
  push:
    branches: [main]

jobs:
  terraform:
    runs-on: ubuntu-latest
    permissions:
      id-token: write       # Required for OIDC → Vault JWT auth
      contents: read
      pull-requests: write
    steps:
      - uses: actions/checkout@v7

      - name: Retrieve Dynatrace credentials from Vault
        uses: hashicorp/vault-action@v4
        with:
          url: ${{ secrets.VAULT_ADDR }}
          method: jwt
          role: dynatrace-deployer
          secrets: |
            secret/data/dynatrace/prod platform_token | DYNATRACE_PLATFORM_TOKEN ;
            secret/data/dynatrace/prod api_token | DYNATRACE_API_TOKEN ;
            secret/data/dynatrace/prod env_url | DYNATRACE_ENV_URL

      - name: Setup Terraform
        uses: hashicorp/setup-terraform@v4

      - name: Terraform Init
        run: terraform init

      - name: Terraform Plan
        run: terraform plan -no-color -out=tfplan
        env:
          DYNATRACE_HTTP_OAUTH_PREFERENCE: "true"

      - name: Terraform Apply
        if: github.ref == 'refs/heads/main' &amp;amp;&amp;amp; github.event_name == 'push'
        run: terraform apply -auto-approve
        env:
          DYNATRACE_HTTP_OAUTH_PREFERENCE: "true"
```

Vault path structure per team per tenant:

```
secret/
└── dynatrace/
    ├── dev/
    │   ├── platform_token    # dt0s16 for dev tenant
    │   ├── api_token         # dt0c01 for dev tenant
    │   └── env_url           # https://{dev-tenant}.live.dynatrace.com
    ├── prod/
    │   ├── platform_token
    │   ├── api_token
    │   └── env_url
    └── teams/
        ├── payments/         # Team-specific OAuth clients
        │   ├── client_id
        │   └── client_secret
        └── platform/
            ├── client_id
            └── client_secret
```

> **What Vault adds — and what it doesn't.** Vault moves the Dynatrace token out of the CI platform's secret store and logs each read. It does not make the token short-lived: `secret/data/…` is a KV v2 path, and the KV engine is *"a generic key-value store used to store arbitrary secrets"* — the Dynatrace token keeps whatever expiry you gave it when you created it. Only Vault's own login token is short-lived. For per-job expiry of the Dynatrace token itself, combine Vault with the minting pattern in §11.
>
> **Vault's JWT role and GitHub's `sub` claim.** If the `dynatrace-deployer` role binds on the token's `sub`, note that per the immutable subject claims announcement (GitHub changelog), *"All repositories created after July 15, 2026 will automatically use the new immutable subject claim format"* (for example `repo:octocat@123456/my-repo@456789:ref:refs/heads/main`); renames and transfers after that date switch too, and existing repositories change only if you opt in. A role bound to the old `repo:<owner>/<repo>:…` form silently stops matching for those repositories.
>
> **Sources:** KV secrets engine (HashiCorp), Immutable subject claims for GitHub Actions OIDC tokens (GitHub changelog).

Policy-as-Code Gates (OPA / Sentinel)

Add governance checks to your pipeline using **OPA/Conftest** (open-source) or **Sentinel** (Terraform Enterprise/Cloud).

OPA/Conftest — Resource Type Allowlist

Restrict which Dynatrace resource types a team can create:

policy/allowed_resources.rego:

```
package main

# Only allow these Dynatrace resource types
allowed_resources := {
  "dynatrace_alerting",
  "dynatrace_autotag_v2",
  "dynatrace_management_zone_v2",
  "dynatrace_maintenance"
}

# resource_changes is flat across the root module and every child module;
# planned_values.root_module.resources would miss anything declared in a module
deny contains msg if {
  some rc in input.resource_changes
  rc.mode == "managed"
  startswith(rc.type, "dynatrace_")
  creates_or_updates(rc)
  not allowed_resources[rc.type]
  msg := sprintf("Resource type '%s' (%s) is not in the allowlist", [rc.type, rc.address])
}

# Deleting a disallowed resource is cleanup, not a violation
creates_or_updates(rc) if {
  some action in rc.change.actions
  action in {"create", "update"}
}
```

> **Provider version note (v1.98.0, June 2026):** `dynatrace_maintenance` in the allowlist above is **deprecated** in provider v1.98+ in favor of `dynatrace_maintenance_windows`. The allowlist stays valid for older provider pins; on v1.98+ add (or substitute) `"dynatrace_maintenance_windows"` so newly-authored maintenance windows pass the policy.
>
> **Read the allowlist above as a mechanism demonstration, not a starting allowlist.** Every one of its four entries resolves to a Settings 2.0 schema marked **Blocked at upgrade** in AUTOM-02's catalog and listed on Dynatrace's Settings 2.0 schemas that are removed in Latest Dynatrace (DT docs):
>
> | Resource | Schema | Upgrade status |
> |---|---|---|
> | `dynatrace_alerting` | `builtin:alerting.profile` | Blocked |
> | `dynatrace_autotag_v2` | `builtin:tags.auto-tagging` | Blocked |
> | `dynatrace_management_zone_v2` | `builtin:management-zones` | Blocked |
> | `dynatrace_maintenance` | `builtin:alerting.maintenance-window` | Blocked |
> | `dynatrace_maintenance_windows` | `builtin:maintenance-windows` | Carries forward |
>
> A governance gate whose allowlist contains only blocked resource types inverts its own purpose: it permits exactly the configuration that stops working when the tenant upgrades, and denies the platform-native resources that survive. Note the useful coincidence in the last two rows — the provider-deprecation fix above (`dynatrace_maintenance` → `dynatrace_maintenance_windows`) happens to move you onto the surviving schema too, for an unrelated reason.
>
> Before copying this policy, replace the set with the resource types **your** teams should actually be creating, and check each one's schema against AUTOM-02's `Upgrade status` column first.

OPA/Conftest — Blocking Resources That Do Not Survive the Upgrade

The more useful policy during a Classic → Gen3 migration is the inverse: fail the build when someone adds a resource type that will stop working. This catches new classic configuration entering the repo while the migration is still in flight — the

failure mode that quietly extends a migration indefinitely.

policy/blocked_at_upgrade.rego:

```
package main

# Terraform resource types whose Settings 2.0 schema is removed when the
# tenant upgrades to the latest Dynatrace. Verified against the provider's
# own resource docs; re-check against AUTOM-02's catalog before relying on it.
blocked_at_upgrade := {
  "dynatrace_alerting",              # builtin:alerting.profile
  "dynatrace_autotag_v2",            # builtin:tags.auto-tagging
  "dynatrace_management_zone_v2",    # builtin:management-zones
  "dynatrace_maintenance",           # builtin:alerting.maintenance-window
  "dynatrace_metric_events",         # builtin:anomaly-detection.metric-events
  "dynatrace_slo_v2"                 # builtin:monitoring.slo
}
# Successors that carry forward: dynatrace_maintenance_windows, dynatrace_davis_anomaly_detectors
# (for metric events), dynatrace_platform_slo (for SLOs), dynatrace_automation_workflow (for alerting)

deny contains msg if {
  some rc in input.resource_changes
  rc.mode == "managed"
  blocked_at_upgrade[rc.type]
  # New resources only: existing classic config can still be maintained mid-migration
  "create" in rc.change.actions
  msg := sprintf(
    "'%s' (%s) uses a Settings schema that is removed at upgrade to the latest Dynatrace — see AUTOM-02 for
the replacement path",
    [rc.type, rc.address]
  )
}
```

Run it as a **warning** rather than a hard failure while you still have classic configuration to maintain — `conftest test --policy policy/ --output table` reports without failing the build when the rule is named `warn` instead of `deny` (`warn contains msg if { … }`). Flip it to `deny` once the migration for that domain is complete, and it becomes a ratchet that stops the classic surface coming back.

OPA/Conftest — Mandatory Team Tagging

Ensure all resources include team ownership metadata:

policy/mandatory_tags.rego:

```
package main

# Resource types that carry a `tags` set must include an owner:&lt;team&gt; tag
tagged_types := {"dynatrace_platform_slo"}

# change.after is the planned state of each created or updated resource,
# in the root module or any child module
deny contains msg if {
  some rc in input.resource_changes
  tagged_types[rc.type]
  rc.change.after != null
  not has_owner_tag(rc.change.after)
  msg := sprintf("'%s' must carry an owner:&lt;team&gt; tag", [rc.address])
}

has_owner_tag(after) if {
  some tag in after.tags
  startswith(tag, "owner:")
}

# A Davis problem trigger with no owner entity tag matches every entity in the
# environment — one team's route becomes an all-hands page
deny contains msg if {
  some rc in input.resource_changes
  rc.type == "dynatrace_automation_workflow"
  rc.change.after != null
  trigger := rc.change.after.trigger[_].event[_].config[_].davis_problem[_]
  not trigger.entity_tags.owner
  msg := sprintf("'%s' has a Davis problem trigger with no owner entity tag", [rc.address])
}
```

Pipeline Integration

```
# Add after terraform plan step
- name: Install Conftest
  run: |
    # Pin the version — "latest" changed the default Rego syntax under existing policies once already
    CONFTEST_VERSION=0.70.0
    wget -q "https://github.com/open-policy-
agent/conftest/releases/download/v${CONFTEST_VERSION}/conftest_${CONFTEST_VERSION}_Linux_x86_64.tar.gz"
    tar xzf "conftest_${CONFTEST_VERSION}_Linux_x86_64.tar.gz"
    sudo mv conftest /usr/local/bin/

- name: Run Policy Checks
  run: |
    terraform show -json tfplan &gt; tfplan.json
    conftest test tfplan.json --policy policy/
```

> **The policies above use Rego v1 syntax** (`deny contains msg if { … }` , `if` on rule bodies). conftest switched its default in v0.60.0 — per its v0.60.0 release notes (GitHub): *"We have set the default version of Rego syntax to v1. This is a breaking change if your Rego policies are not compatible with the v1 syntax."* Older `deny[msg] { … }` policies fail to parse on current conftest unless you pass `--rego-version v0` .
>
> **Read `resource_changes` , not `planned_values.root_module.resources` .** Per the Terraform JSON output format (HashiCorp), resources declared in modules sit under `root_module.child_modules[*]` , so a policy that reads only `root_module.resources` silently passes everything in a module — and AUTOM-09's layout puts Dynatrace resources in modules. `resource_changes` lists every resource in one flat array with its `address` , `type` , `change.actions` and `change.after` , including deletes. The three policies above were run with conftest 0.70.0 against a sample plan

> containing root-module and child-module resources (10/02/2026) — still run `conftest test` against one of your own plans before adopting them.

Sentinel (HCP Terraform / Terraform Enterprise)

For teams on **HCP Terraform** (formerly Terraform Cloud) or **Terraform Enterprise**, Sentinel policies enforce governance at the workspace level:

```
# sentinel/restrict_resource_types.sentinel
import "tfplan/v2" as tfplan

# Platform-native types that carry forward at upgrade. The classic trio earlier
# versions of this example allowed (dynatrace_alerting, dynatrace_autotag_v2,
# dynatrace_management_zone_v2) is blocked at upgrade — see the Conftest note above.
allowed_types = [
  "dynatrace_automation_workflow",
  "dynatrace_document",
  "dynatrace_segment",
  "dynatrace_platform_slo",
  "dynatrace_davis_anomaly_detectors",
  "dynatrace_maintenance_windows",
]

main = rule {
  all tfplan.resource_changes as _, rc {
    rc.type in allowed_types or not (rc.type matches "dynatrace_.*")
  }
}
```

> **Unlike the first Conftest allowlist, these six types carry forward at upgrade** — none of their schemas is on the removed-schemas list. Still adjust the list to what your teams actually create.
>
> **Important — Sentinel Limitations:** Sentinel evaluates Terraform **plans** — it does not grant or restrict Dynatrace API access at runtime. It cannot reduce the permissions of a token that has broad scope. If a pipeline token can write all Synthetic monitors, Sentinel cannot narrow that. Use **Dynatrace IAM policies** (see **AUTOM-04: IAM Policy Management**) for platform-level access control, and Sentinel for pipeline-level governance. Sentinel runs only in **HCP Terraform** and **Terraform Enterprise** — it is not available in open-source Terraform, GitHub Actions, or plain CI runners. For those environments use **OPA/Conftest** as shown above; HCP Terraform can also run OPA policy sets natively — its policy enforcement overview (HashiCorp) lists *"OPA: Open Policy Agent with Rego language"* among its frameworks.

Drift Detection Workflow

Schedule automatic drift detection to catch manual UI changes ("ClickOps") that bypass your Terraform pipeline:

```yaml
name: Drift Detection

on:
  schedule:
    - cron: '0 6 * * 1-5'   # Every weekday at 6am UTC
  workflow_dispatch: {}       # Allow manual trigger

jobs:
  detect-drift:
    runs-on: ubuntu-latest
    permissions:
      contents: read
      issues: write      # the restricted default token cannot open issues
    steps:
      - uses: actions/checkout@v7

      - name: Setup Terraform
        uses: hashicorp/setup-terraform@v4
        with:
          # The default wrapper exits 0 when terraform exits 2, so PIPESTATUS[0]
          # below would never see the drift code
          terraform_wrapper: false

      - name: Terraform Init
        run: terraform init

      - name: Check for Drift
        id: drift
        run: |
          set +e
          terraform plan -detailed-exitcode -no-color 2&gt;&amp;1 | tee drift-output.txt
          # $? here would be tee's exit code (always 0); PIPESTATUS[0] is terraform's
          echo "exitcode=${PIPESTATUS[0]}" &gt;&gt; "$GITHUB_OUTPUT"
        env:
          DYNATRACE_ENV_URL: ${{ secrets.DT_ENV_URL }}
          DYNATRACE_PLATFORM_TOKEN: ${{ secrets.DT_PLATFORM_TOKEN }}
          DYNATRACE_API_TOKEN: ${{ secrets.DT_API_TOKEN }}
          DYNATRACE_HTTP_OAUTH_PREFERENCE: "true"

      - name: Create Issue on Drift
        if: steps.drift.outputs.exitcode == '2'
        uses: actions/github-script@v9
        with:
          script: |
            const fs = require('fs');
            const drift = fs.readFileSync('drift-output.txt', 'utf8').slice(-3000);
            await github.rest.issues.create({
              owner: context.repo.owner,
              repo: context.repo.repo,
              title: '⚠️ Dynatrace configuration drift detected',
              body: `Scheduled drift detection found changes not in Terraform
state.\n\n\`\`\`\n${drift}\n\`\`\`\n\nSomeone may have made manual changes in the Dynatrace UI.`,
              labels: ['drift', 'dynatrace']
            });
```

> **Exit codes:** `terraform plan -detailed-exitcode` returns `0` = no changes, `1` = error, `2` = drift detected. This lets your pipeline take different actions based on the result. Read it from `${PIPESTATUS[0]}`, not `$?`: after `… | tee`, `$?` is `tee`'s status, so the drift step would always report `0` and the issue would never be opened — a silent green. (GitHub's default `bash` shell supports `PIPESTATUS`.)
>
> **Why `terraform_wrapper: false`.** `setup-terraform` installs a wrapper script as `terraform` unless told not to — the input *"Defaults to `true`"*. In the v4.0.1 wrapper source (HashiCorp GitHub), `if (exitCode === 0 || exitCode === 2)`

> `{` returns without failing, so the process exits `0` on drift. With the wrapper in place `PIPESTATUS[0]` is `0`, the issue step never runs, and drift goes unreported — the same silent green as the `$?` bug, by a different route. (The alternative is to keep the wrapper, drop the `tee` / `echo`, and read the wrapper's own `steps.drift.outputs.exitcode`.)
>
> **Why the `permissions:` block.** Under the restricted default, `GITHUB_TOKEN` gets *"just read access for the contents and packages permissions"* (Managing GitHub Actions settings (GitHub Docs)), so `issues.create` returns 403 and the drift is found but never reported.

Reusable GitHub Actions Workflows

For organizations managing multiple Dynatrace tenants or LOB repositories, create **reusable workflows** that any repo can call:

.github/workflows/terraform-dynatrace.yml (in a shared workflow repo):

```yaml
name: Terraform Dynatrace (Reusable)

on:
  workflow_call:
    inputs:
      working_directory:
        required: false
        type: string
        default: '.'
      terraform_version:
        required: false
        type: string
        default: '1.14.5'   # pin what you tested; S3 use_lockfile needs 1.11+
    secrets:
      DT_ENV_URL:
        required: true
      DT_PLATFORM_TOKEN:
        required: true
      DT_API_TOKEN:
        required: true

jobs:
  plan:
    runs-on: ubuntu-latest
    permissions:
      contents: read
      pull-requests: write
    defaults:
      run:
        working-directory: ${{ inputs.working_directory }}
    steps:
      - uses: actions/checkout@v7

      - name: Setup Terraform
        uses: hashicorp/setup-terraform@v4
        with:
          terraform_version: ${{ inputs.terraform_version }}

      - name: Terraform Init
        run: terraform init

      - name: Terraform Plan
        run: terraform plan -no-color -out=tfplan
        env:
          DYNATRACE_ENV_URL: ${{ secrets.DT_ENV_URL }}
          DYNATRACE_PLATFORM_TOKEN: ${{ secrets.DT_PLATFORM_TOKEN }}
          DYNATRACE_API_TOKEN: ${{ secrets.DT_API_TOKEN }}
          DYNATRACE_HTTP_OAUTH_PREFERENCE: "true"

      - name: Post Plan to PR
        if: github.event_name == 'pull_request'
        uses: borchero/terraform-plan-comment@v3
        with:
          token: ${{ github.token }}
          planfile: tfplan
          # defaults.run applies only to run: steps; the action defaults to "."
          working-directory: ${{ inputs.working_directory }}

  apply:
    needs: plan
    if: github.ref == 'refs/heads/main' &amp;amp;&amp;amp; github.event_name == 'push'
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: ${{ inputs.working_directory }}
```

```
    steps:
      - uses: actions/checkout@v7

      - name: Setup Terraform
        uses: hashicorp/setup-terraform@v4
        with:
          terraform_version: ${{ inputs.terraform_version }}

      - name: Terraform Init
        run: terraform init

      - name: Terraform Apply
        run: terraform apply -auto-approve
        env:
          DYNATRACE_ENV_URL: ${{ secrets.DT_ENV_URL }}
          DYNATRACE_PLATFORM_TOKEN: ${{ secrets.DT_PLATFORM_TOKEN }}
          DYNATRACE_API_TOKEN: ${{ secrets.DT_API_TOKEN }}
          DYNATRACE_HTTP_OAUTH_PREFERENCE: "true"
```

Calling the reusable workflow from a LOB repo:

```
# .github/workflows/deploy.yml (in dt-lob-payments repo)
name: Deploy Dynatrace Config

on:
  pull_request:
    branches: [main]
  push:
    branches: [main]

jobs:
  terraform:
    uses: my-org/dt-templates/.github/workflows/terraform-dynatrace.yml@main
    with:
      working_directory: terraform/
    secrets:
      DT_ENV_URL: ${{ secrets.DT_ENV_URL }}
      DT_PLATFORM_TOKEN: ${{ secrets.DT_PLATFORM_TOKEN }}
      DT_API_TOKEN: ${{ secrets.DT_API_TOKEN }}
```

> **Plan comment working directory.** `defaults.run.working-directory` sets *"a default working directory for all run steps"* (Workflow syntax (GitHub Docs)) — it does not reach `uses:` steps. `terraform-plan-comment` has its own `working-directory` input (default `"."`), so it is passed explicitly above; without it the action looks for `tfplan` in the repository root.
>
> **Multi-tenant pattern:** For organizations with 5+ tenants, the reusable workflow is called once per environment with different secrets, providing a consistent deployment experience across all tenants.

---

4. GitLab CI/CD

Pipeline Configuration

.gitlab-ci.yml:

```
stages:
  - validate
  - deploy-staging
  - deploy-production

.monaco-setup: &amp;monaco-setup
  before_script:
    - curl -L https://github.com/Dynatrace/dynatrace-configuration-as-code/releases/latest/download/monaco-linux-amd64 -o monaco
    - chmod +x monaco
    - mv monaco /usr/local/bin/

validate:
  stage: validate
  &lt;&lt;: *monaco-setup
  script:
    - monaco deploy manifest.yaml --environment development --dry-run
  rules:
    - if: $CI_PIPELINE_SOURCE == "merge_request_event"

deploy-staging:
  stage: deploy-staging
  &lt;&lt;: *monaco-setup
  script:
    - monaco deploy manifest.yaml --environment staging
  environment:
    name: staging
  rules:
    - if: $CI_COMMIT_BRANCH == "main"

deploy-production:
  stage: deploy-production
  &lt;&lt;: *monaco-setup
  script:
    - monaco deploy manifest.yaml --environment production
  environment:
    name: production
  rules:
    - if: $CI_COMMIT_BRANCH == "main"
      when: manual
```

Variables Configuration

In GitLab: **Settings** → **CI/CD** → **Variables**

Variable	Protected	Masked
`DT_DEV_URL`	No	No
`DT_DEV_TOKEN`	No	Yes
`DT_STAGING_URL`	No	No
`DT_STAGING_TOKEN`	Yes	Yes
`DT_PROD_URL`	Yes	No
`DT_PROD_TOKEN`	Yes	Yes

---

5. Bitbucket Pipelines

Bitbucket Cloud (bitbucket.org) ships **Bitbucket Pipelines** as the integrated CI/CD service. The pipeline definition lives in a `bitbucket-pipelines.yml` file at the repository root. Variables (including secrets) live at three scopes: workspace, repository, and deployment-environment.

> **Bitbucket Data Center / Server** uses a different mechanism (Bitbucket Pipelines is a Cloud-only feature). For self-hosted Bitbucket, integrate via Jenkins, GitLab CI, or another CI/CD tool — the patterns below assume Bitbucket Cloud.
>
> ### Provider landscape note (mid-2026)
>
> The Terraform-provider landscape for Bitbucket Cloud is in active transition:
>
> - `DrFaust92/bitbucket` — the long-standing community provider — entered **maintenance mode** in March 2026. The maintainer is no longer using Bitbucket and has limited bandwidth for substantive changes.
> - **Bitbucket App Passwords are retired.** Atlassian's app-password deprecation announcement (Atlassian) states that *"Integrations with app passwords will stop working entirely on June 9, 2026."* Existing `DrFaust92` configurations need an API token (Atlassian) in place of the App Password (workaround community-validated: the Atlassian-account email as `BITBUCKET_USERNAME` + an API token as `BITBUCKET_PASSWORD`, with `-parallelism=2` to avoid throttling).
> - `FabianSchurig/bitbucket` — a newer, actively-maintained provider — is the going-forward recommendation. It uses a "generic resources" architecture (resources expose Bitbucket Cloud API endpoints directly via typed core fields, and most also take `request_body` for full API flexibility) and authenticates with an Atlassian API token or a workspace/repository access token. The current release is v0.20.2 (08/21/2026) — **pre-1.0, with schema flux possible** — so verify the examples below against the provider docs at use time.
>
> The examples in this section use `FabianSchurig/bitbucket`. For teams with existing `DrFaust92` code, the migration guide covers resource-name mapping, auth changes, and path-parameter renames (`owner` → `workspace`, `repository` → `repo_slug`).

Pipeline — Monaco Deploy

bitbucket-pipelines.yml at repo root:

```
image: atlassian/default-image:4

pipelines:
  pull-requests:
    '**':
      - step:
          name: Validate Monaco config
          script:
            # Anchor the install LINE, not the step: a step's own script: key replaces
            # a merged one, and every step runs in a fresh container, so each step
            # has to install Monaco itself.
            - &amp;install-monaco curl -L https://github.com/Dynatrace/dynatrace-configuration-as-
code/releases/latest/download/monaco-linux-amd64 -o /usr/local/bin/monaco &amp;&amp; chmod +x
/usr/local/bin/monaco
            - monaco deploy manifest.yaml --environment development --dry-run

  branches:
    main:
      - step:
          name: Deploy to staging
          deployment: staging
          script:
            - *install-monaco
            - monaco deploy manifest.yaml --environment staging

      - step:
          name: Deploy to production
          deployment: production
          trigger: manual
          script:
            - *install-monaco
            - monaco deploy manifest.yaml --environment production
```

> **Why not a step anchor?** The common `- step: &install-monaco` + `<<: *install-monaco` pattern does not work here: with YAML merge keys, a key set in the mapping overrides the merged one, so each step's own `script:` replaces the install lines and every step fails with `monaco: command not found`. Anchoring the single install line and referencing it inside each `script:` list keeps the install in every step.

Each step declared with `deployment: <env>` inherits that environment's deployment variables. The `trigger: manual` on the production step gates the deploy behind a manual confirmation in the Bitbucket UI.

Pipeline — Terraform with Combined Auth

Mirrors the GitHub Actions Terraform pattern (§3 *Terraform with Combined Auth*) — Platform Token + API Token together for full Dynatrace resource coverage:

```
image: hashicorp/terraform:1.14

pipelines:
  pull-requests:
    '**':
      - step:
          name: Terraform Plan
          deployment: staging
          script:
            - terraform init
            - terraform plan -no-color -out=tfplan
            - terraform show -no-color tfplan &gt; plan.txt
          artifacts:
            - plan.txt
            - tfplan

  branches:
    main:
      - step:
          name: Terraform Apply (staging)
          deployment: staging
          script:
            - terraform init
            - terraform apply -auto-approve

      - step:
          name: Terraform Apply (production)
          deployment: production
          trigger: manual
          script:
            - terraform init
            - terraform apply -auto-approve
```

The pipeline reads `DYNATRACE_ENV_URL`, `DYNATRACE_PLATFORM_TOKEN`, `DYNATRACE_API_TOKEN`, and `DYNATRACE_HTTP_OAUTH_PREFERENCE` from the deployment environment's variables — see Variables and Secrets below.

Terraform Bitbucket Provider — FabianSchurig (recommended for new code)

The **`FabianSchurig/bitbucket`** provider manages Bitbucket Cloud resources via the Cloud API. Architecture quirks worth knowing before reading the example:

- **Typed core fields + `request_body` escape hatch.** Each resource has a few typed arguments for the most-common fields (`workspace`, `repo_slug`, `key`, `secured`, etc.), and most also take a `request_body` JSON string for full API payload flexibility — when a field you need is not surfaced as a typed argument, you set it via `request_body = jsonencode({...})`. Not every resource has it: `bitbucket_commit_file` exposes only `workspace`, `repo_slug`, `commit` and `path`.
- **Boolean fields are strings.** `secured = "true"`, `enabled = "true"` — typed as string, not bool. The generic-resource architecture preserves API string representations.
- **Auth: Atlassian API token or access token.** The provider *"authenticates via HTTP Basic Auth using an Atlassian API token"*, so set `BITBUCKET_USERNAME` (your Atlassian account email) **and** `BITBUCKET_TOKEN` (the API token). A workspace or repository access token needs `BITBUCKET_TOKEN` alone. No App Password.

Key resources used in this section:

Resource	What it manages
`bitbucket_repos`	The repo itself (CRUD); core fields + `request_body` for less-common settings
`bitbucket_pipeline_config`	Enables/disables Bitbucket Pipelines on the repo (`enabled = "true"`)

Resource	What it manages
`bitbucket_repo_settings`	Repository settings split out of the legacy repo resource
`bitbucket_deployments`	A deployment environment (named — staging / production / etc.)
`bitbucket_deployment_variables`	Variable scoped to a deployment environment
`bitbucket_pipeline_variables`	Repository-scoped variable (formerly `bitbucket_repository_variable` in DrFaust92)
`bitbucket_workspace_pipeline_variables`	Workspace-scoped variable shared across repos
`bitbucket_commit_file`	Wraps `POST /repositories/{workspace}/{repo_slug}/src` , but its schema has no argument for file content, branch or message — it cannot commit `bitbucket-pipelines.yml` (see below)
`bitbucket_pipeline_schedules`	Scheduled pipeline runs

Combined Workspace — Bitbucket and Dynatrace in One Apply

The load-bearing pattern: a single Terraform workspace provisions the Bitbucket repo, enables pipelines on it, creates deployment environments, sets the variables holding Dynatrace credentials, **and** stands up the Dynatrace configuration the pipeline will deploy. One `terraform apply` brings up the whole CI/CD-to-Dynatrace path except the `bitbucket-pipelines.yml` file itself, which goes in with the repository's first push (see *Ordering* below).

Why this is useful: for net-new tenants or new app teams, you can codify the entire onboarding (repo, pipeline, Dynatrace tenant config) in a single artifact. The drift-detection story is also unified — one `terraform plan` shows drift in both Bitbucket and Dynatrace.

> **Verify against current provider docs at use time.** The FabianSchurig provider is pre-1.0. The configuration below passes `terraform validate` against FabianSchurig/bitbucket v0.20.2 and dynatrace-oss/dynatrace v1.105.0 (10/02/2026) — confirm against the registry docs before applying a newer version.

main.tf:

```
terraform {
  required_providers {
    bitbucket = {
      source  = "FabianSchurig/bitbucket"
      version = "~&gt; 0.20.2"  # Pre-1.0; pin tightly until stable 1.x lands
    }
    dynatrace = {
      source  = "dynatrace-oss/dynatrace"
      version = "~&gt; 1.88"
    }
  }
}

# Bitbucket provider — Atlassian API token (App Passwords are retired)
provider "bitbucket" {
  # Atlassian API token: BITBUCKET_USERNAME (account email) + BITBUCKET_TOKEN.
  # Workspace/repository access token: BITBUCKET_TOKEN alone.
}

# Dynatrace provider — combined auth (Platform Token + API Token)
provider "dynatrace" {
  dt_env_url   = var.dt_env_url
  dt_api_token = var.dt_api_token
  # Platform Token is read from DT_PLATFORM_TOKEN env var by the provider
}

variable "workspace" { type = string }
variable "dt_env_url" { type = string }
variable "dt_api_token" {
  type      = string
  sensitive = true
}
variable "dt_platform_token" {
  type      = string
  sensitive = true
}

# --- 1. Bitbucket repo ---
resource "bitbucket_repos" "config_repo" {
  workspace = var.workspace
  repo_slug = "dynatrace-config"

  # Less-common settings go through request_body in the generic-resource architecture
  request_body = jsonencode({
    is_private  = true
    fork_policy = "no_forks"
    description = "Monaco / Terraform configs for Dynatrace tenant"
  })
}

# --- 2. Enable Bitbucket Pipelines on the repo ---
resource "bitbucket_pipeline_config" "config_repo_pipelines" {
  workspace = bitbucket_repos.config_repo.workspace
  repo_slug = bitbucket_repos.config_repo.repo_slug
  enabled   = "true"  # String, not bool — the generic schema preserves API representations
}

# --- 3. Deployment environments ---
resource "bitbucket_deployments" "staging" {
  workspace = var.workspace
  repo_slug = bitbucket_repos.config_repo.repo_slug
  name      = "staging"
}

resource "bitbucket_deployments" "production" {
```

```
  workspace = var.workspace
  repo_slug = bitbucket_repos.config_repo.repo_slug
  name      = "production"
}

# --- 4. Secured variables holding Dynatrace credentials ---
# Repository-scoped (visible to every step):
resource "bitbucket_pipeline_variables" "dt_env_url" {
  workspace = var.workspace
  repo_slug = bitbucket_repos.config_repo.repo_slug
  key       = "DYNATRACE_ENV_URL"
  value     = var.dt_env_url
  secured   = "false"  # String
}

# Without this the Dynatrace provider falls back to the API token for everything
resource "bitbucket_pipeline_variables" "dt_oauth_preference" {
  workspace = var.workspace
  repo_slug = bitbucket_repos.config_repo.repo_slug
  key       = "DYNATRACE_HTTP_OAUTH_PREFERENCE"
  value     = "true"
  secured   = "false"
}

# Per-environment (different token per env — locked to that deployment):
resource "bitbucket_deployment_variables" "staging_platform_token" {
  workspace        = var.workspace
  repo_slug        = bitbucket_repos.config_repo.repo_slug
  environment_uuid = bitbucket_deployments.staging.uuid
  key              = "DYNATRACE_PLATFORM_TOKEN"
  value            = var.dt_platform_token  # In real use, source per-env tokens separately
  secured          = "true"
}

resource "bitbucket_deployment_variables" "staging_api_token" {
  workspace        = var.workspace
  repo_slug        = bitbucket_repos.config_repo.repo_slug
  environment_uuid = bitbucket_deployments.staging.uuid
  key              = "DYNATRACE_API_TOKEN"
  value            = var.dt_api_token
  secured          = "true"
}

resource "bitbucket_deployment_variables" "prod_platform_token" {
  workspace        = var.workspace
  repo_slug        = bitbucket_repos.config_repo.repo_slug
  environment_uuid = bitbucket_deployments.production.uuid
  key              = "DYNATRACE_PLATFORM_TOKEN"
  value            = var.dt_platform_token
  secured          = "true"
}

resource "bitbucket_deployment_variables" "prod_api_token" {
  workspace        = var.workspace
  repo_slug        = bitbucket_repos.config_repo.repo_slug
  environment_uuid = bitbucket_deployments.production.uuid
  key              = "DYNATRACE_API_TOKEN"
  value            = var.dt_api_token
  secured          = "true"
}

# --- bitbucket-pipelines.yml is NOT managed here ---
# bitbucket_commit_file has no argument for file content, branch or commit
# message (v0.20.2 schema: workspace, repo_slug, commit, path). Commit the
# pipeline file with the repository's first push, after this apply has enabled
```

```
# pipelines and created the variables.

# --- 5. Dynatrace resources the pipeline will deploy on top of ---
# (Bootstrap resources — things you want present before the pipeline runs for
# the first time. Day-to-day app-team configs land in the repo and deploy via
# the pipeline itself.)
resource "dynatrace_segment" "platform_baseline" {
  name        = "platform-baseline"
  description = "Data from the platform-baseline host group"
  is_public   = true

  includes {
    items {
      data_object = "_all_data_object"
      # The filter is the segment editor's JSON filter tree, not DQL. Build the
      # segment once in the UI and export it (AUTOM-04 §4 Segment) rather than
      # hand-writing the tree.
      filter = file("${path.module}/segments/platform-baseline.json")
    }
  }
}
# Classic equivalent on a tenant not yet upgraded: a dynatrace_management_zone_v2
# with a HOST_GROUP_NAME rule. Management zones are blocked at upgrade — the
# segment above carries the filtering job, IAM policies carry the access job.
```

Ordering is load-bearing:

- `bitbucket_repos` must exist before `bitbucket_pipeline_config`, `bitbucket_pipeline_variables`, or `bitbucket_deployments` referencing it.
- `bitbucket_deployments` must exist before any `bitbucket_deployment_variables` referencing it (the `environment_uuid` is the API-returned identifier).
- **Push `bitbucket-pipelines.yml` only after the apply has finished.** If the file lands in the repo before pipelines are enabled and the variables exist, the first push triggers a pipeline run that fails (either because pipelines aren't active, or because authentication has nothing to read). The provider cannot sequence this for you: `bitbucket_commit_file` takes no file content. If the commit has to be part of the same automation, call Bitbucket's `POST /repositories/{workspace}/{repo_slug}/src` endpoint from a `terraform_data` resource with a `local-exec` provisioner that `depends_on` the variables — a step outside the provider that you own and test yourself.

Caveat on per-env tokens: the example above uses a single `var.dt_platform_token` for both staging and production deployment variables. In a real setup, you'd want **separate Dynatrace tokens per environment** — typically by running the staging and production applies in separate workspaces, or by sourcing per-environment tokens from a secrets manager (Vault, AWS Secrets Manager, Bitbucket OIDC-to-Vault flow).

Variables and Secrets

Bitbucket Pipelines has three variable scopes. Pick the right scope per credential:

Scope	Terraform resource	When to use
Workspace	`bitbucket_workspace_pipeline_variables`	Credentials shared across many repos in the workspace (e.g., a workspace-wide Dynatrace token if you don't separate by app).
Repository	`bitbucket_pipeline_variables`	Per-repo credentials — typical for a `dynatrace-config` repo with its own Dynatrace tokens.
Deployment environment	`bitbucket_deployment_variables`	Per-environment credentials (staging vs. production tokens). The recommended scope when you want different Dynatrace tokens (or different tenants) per environment.

Variable	Scope	`secured`	Why
`DYNATRACE_ENV_URL`	Repository	No	Tenant URL is not secret; visible in every step
`DYNATRACE_PLATFORM_TOKEN`	Deployment	Yes	Per-env; write-only after creation
`DYNATRACE_API_TOKEN`	Deployment	Yes	Per-env; covers synthetics and classic SLOs, which need a classic API token
`DYNATRACE_HTTP_OAUTH_PREFERENCE`	Repository	No	Static flag, not a credential; created by `bitbucket_pipeline_variables.dt_oauth_preference` above

`secured = "true"` makes the variable value write-only — once set, the Bitbucket API won't return it (and Terraform won't drift-detect on the value). If you need to rotate, change the Terraform value and re-apply.

> **Note on YAML templating and secured variables.** Bitbucket Pipelines has a `${{ }}` YAML-templating mechanism that injects workspace and repository variables (plus a few `BITBUCKET_*` defaults) into the pipeline YAML — and the docs list as a limitation that *"Secured variables and runtime-defined custom variables are not supported."* A Dynatrace token stored in a secured variable therefore cannot be rendered into a pipeline YAML field; read secured variables only via environment-variable substitution inside `script:` blocks. A related addition: **shared pipeline variables** let a step export variables to subsequent steps in the same pipeline, with a 50-variable / 100KB limit.
>
> **Sources:** Variables and secrets (Atlassian) — three variable scopes (workspace / repository / deployment) and precedence; shared pipeline variables; YAML templating limits.

OIDC Note

Bitbucket Pipelines supports OIDC for federating to AWS, GCP, and Vault — see the Atlassian docs on Integrate Pipelines with resource servers using OIDC. The pattern is to add `oidc: true` at the pipeline step level and configure the resource server (AWS / GCP / Vault) to trust the Bitbucket OIDC issuer.

Preview — Dynatrace workload identity federation. Dynatrace now documents Workload identity federation (DT docs) as a **Preview** feature (*"Workload identity federation is an Early Access feature."*): *"Workload identity federation (WIF) lets an external workload call the Dynatrace API with a token issued by its own identity provider."* A trust policy (issuer URL + audience) and a service-user mapping are configured through the Account Management API; the identity provider must sign its tokens with ES256 or RS256, and either publish an OpenID Connect discovery document over HTTPS (URL mode) or have its JSON Web Key Set pasted into the trust policy (manual mode). The page names *"a GitHub Actions job"* among suitable workloads; Bitbucket Pipelines is not named — verify that WIF is available in your account and that your Bitbucket OIDC issuer satisfies the prerequisites before relying on it. Until then, the working pattern remains a *secured variable holding a long-lived token*, rotated on a cadence, or the Vault indirection below.

If you want short-lived credentials anyway, the indirection is: Bitbucket OIDC → Vault (or AWS Secrets Manager) → fetches a Dynatrace token at pipeline start. The pipeline still ends up with a token in memory, but the token in storage is in Vault, not in Bitbucket variables.

> **Sources:**
>
> - Bitbucket provider (FabianSchurig) — actively-maintained successor to `DrFaust92/bitbucket`; v0.20.2 released 08/21/2026 (pre-1.0, schema flux possible).
> - Bitbucket provider docs v0.20.2 (FabianSchurig GitHub) — *"The provider authenticates via HTTP Basic Auth using an Atlassian API token."* Resource schemas (including `commit-file.md`) in the same `docs/` tree.
> - Migration guide DrFaust92 → FabianSchurig — resource-name mapping, auth changes, path-parameter renames.
> - DrFaust92/terraform-provider-bitbucket — long-standing provider, now in maintenance mode (issue #242, March 2026).
> - Get started with Bitbucket Pipelines (Atlassian) — Pipelines overview.

- API tokens (Atlassian) — *"They are the long term replacement for App passwords."*
- App password deprecation (Atlassian) — *"Integrations with app passwords will stop working entirely on June 9, 2026."*
- Integrate Pipelines with resource servers using OIDC (Atlassian) — Bitbucket OIDC for AWS/GCP/Vault.
- Workload identity federation (DT docs) — Preview (Early Access); *"An identity provider that issues tokens signed with ES256 or RS256. The provider must either publish an OpenID Connect discovery document over HTTPS (URL mode), or you must paste its JSON Web Key Set directly when you create the trust policy"*

---

6. Atlassian Bamboo (with Bitbucket + S3)

This section recipes the **Atlassian-only stack** for Terraform GitOps against Dynatrace: a **Bitbucket** repository holding the Terraform code (using the layout from AUTOM-09 §2), **Atlassian Bamboo Data Center** orchestrating the plan/apply pipeline, and **AWS S3** (with S3-native state locking) as the decoupled state backend. The PR review gate lives in Bitbucket; Bamboo applies on merge to long-lived branches.

The pattern targets shops that already license Bamboo and want to keep CI/CD in the same vendor ecosystem as their source-control layer — without adopting Bitbucket Pipelines or a third-party CI service.

Stack Assumed

Component	Choice
CI/CD	Bamboo Data Center (self-hosted; Bamboo Cloud was retired by Atlassian)
Source	Bitbucket Cloud or Bitbucket Data Center, configured as a Linked Repository in Bamboo and referenced with `scope: global`
Plan definition	Bamboo Specs YAML (the modern code-as-config approach over UI-only plans)
State backend	AWS S3 (single bucket, one key per environment), locked with `use_lockfile = true`
AWS auth on Bamboo agent	IAM role attached to the Bamboo agent EC2 instance (preferred); static keys in Bamboo encrypted variables (`BAMSCRT@...`) as fallback
Dynatrace auth	Platform Token + API Token (combined auth — see §3 *Terraform with Combined Auth*) as Bamboo encrypted plan variables
PR review	Bitbucket PRs; required reviewers + branch restrictions configured in Bitbucket

> If your shop runs Bamboo Cloud, this section won't apply — Atlassian retired Bamboo Cloud in 2017. Bamboo Data Center is the only flavor supported today, and Atlassian's documentation site states *"Data Center support ends on March 28, 2029."* — weigh that date if you are choosing a CI platform rather than keeping one.

Plan Specs YAML — Terraform with Combined Auth

The full plan covering plan + apply against one environment. Mirrors the GitHub Actions Combined-Auth pattern (§3 *Terraform with Combined Auth*).

```
---
version: 2
plan:
  project-key: DTRF
  key: TERRAFORM
  name: Dynatrace Terraform — Production
  description: Plan and apply Terraform against the production Dynatrace tenant

repositories:
  # A Linked Repository an admin has already set up in Bamboo. (A `type: bitbucket`
  # + `slug:` block here would instead create a plan-local repository.)
  - dynatrace-terraform:
      scope: global

branches:
  create: for-new-branch
  delete:
    after-deleted-days: 30

stages:
  - Plan:
      jobs:
        - Plan
  - Apply:
      manual: true
      jobs:
        - Apply

Plan:
  artifact-subscriptions: []
  tasks:
    - checkout:
        repository: dynatrace-terraform
    - script:
        # Bamboo exports plan variables as $bamboo_&lt;NAME&gt;; the provider reads DYNATRACE_*
        - export DYNATRACE_ENV_URL="$bamboo_DYNATRACE_ENV_URL"
DYNATRACE_PLATFORM_TOKEN="$bamboo_DYNATRACE_PLATFORM_TOKEN"
DYNATRACE_API_TOKEN="$bamboo_DYNATRACE_API_TOKEN"
DYNATRACE_HTTP_OAUTH_PREFERENCE="$bamboo_DYNATRACE_HTTP_OAUTH_PREFERENCE"
        - cd envs/production
        - terraform init -input=false
        - terraform plan -input=false -no-color -out=tfplan
        - terraform show -no-color tfplan &gt; plan.txt
  artifacts:
    - name: tfplan
      pattern: envs/production/tfplan
      shared: true
    - name: plan.txt
      pattern: envs/production/plan.txt
      shared: true

Apply:
  artifact-subscriptions:
    - artifact: tfplan
  tasks:
    - checkout:
        repository: dynatrace-terraform
    - script:
        - export DYNATRACE_ENV_URL="$bamboo_DYNATRACE_ENV_URL"
DYNATRACE_PLATFORM_TOKEN="$bamboo_DYNATRACE_PLATFORM_TOKEN"
DYNATRACE_API_TOKEN="$bamboo_DYNATRACE_API_TOKEN"
DYNATRACE_HTTP_OAUTH_PREFERENCE="$bamboo_DYNATRACE_HTTP_OAUTH_PREFERENCE"
        - cd envs/production
        - terraform init -input=false
        - terraform apply -input=false -auto-approve tfplan
```

```
variables:
  DYNATRACE_ENV_URL: https://my-tenant.live.dynatrace.com
  # Encrypt via Specs → Sensitive data encryption in the Bamboo UI.
  # The BAMSCRT@... values below are placeholders; generate real ones on your Bamboo server.
  DYNATRACE_PLATFORM_TOKEN: BAMSCRT@0@0@&lt;encrypted-platform-token&gt;
  DYNATRACE_API_TOKEN: BAMSCRT@0@0@&lt;encrypted-api-token&gt;
  DYNATRACE_HTTP_OAUTH_PREFERENCE: "true"
```

Key elements:

- `version: 2` — Bamboo Specs YAML format
- `repositories:` — references a **Linked Repository** already configured globally in Bamboo (admin sets it up once; plans reference it by name with `scope: global`)
- `branches:` — Plan Branches: a child plan is auto-created for every new Bitbucket branch; cleaned up 30 days after branch deletion
- `stages:` — two stages; the `Apply` stage has `manual: true` so it pauses after Plan completes
- `artifact-subscriptions:` on Apply — pulls the `tfplan` artifact produced by Plan, ensuring Apply runs *the plan that was reviewed*, not a re-plan
- `variables:` — plan-scoped values; encrypted variables use the `BAMSCRT@0@0@...` format generated through *Specs → Sensitive data encryption*
- `export … "$bamboo_…"` — Bamboo hands plan variables to script tasks under a `bamboo_` prefix, so without these lines Terraform sees no `DYNATRACE_*` variables and runs with no URL or credentials

PR Workflow with Plan Branches

Bamboo's **Plan Branches** feature creates a child plan per Bitbucket branch automatically. Combined with branch-aware staging, you get the standard plan-on-PR / apply-on-merge GitOps cycle without manual plan configuration per PR.

How it composes:

1. **Developer opens a PR** from `feat/new-segment` → `main` in Bitbucket.
2. **Bamboo detects the new branch** (via the Linked Repository's polling or webhook trigger) and creates a child plan `DTRF-TERRAFORM-FEAT-NEW-SEGMENT` automatically.
3. **The child plan runs `Plan` stage only.** The `Apply` stage is marked `manual: true` and is gated behind a reviewer in Bamboo — but in the PR workflow you typically also wire stage permissions so the gate effectively means "merge the PR first."
4. **PR review happens in Bitbucket.** Required reviewers + branch restrictions are enforced there; Bamboo posts plan output as a build status / Bitbucket build update.
5. **Merge to `main` triggers the main plan.** Plan runs, then the manual Apply gate waits for an authorized user to release production.
6. **Branch deletion** — once the PR is merged and the source branch deleted, the child plan is removed after `delete.after-deleted-days: 30`.

Gating the Apply stage to main only:

In community practice, the usual approach is **branch-aware stage conditions** through plan-branch overrides. Do not use `final: true` for this — Bamboo runs a final stage *"even if previous results fail"*, so it is not a gate. In practice for this pattern:

- Define the Apply stage with `manual: true` in the YAML (as above).
- In the Bamboo UI under the plan's *Branches* tab, set the **Branch overrides** to disable the Apply stage on plan branches — only the main-branch plan executes Apply.
- Alternatively, gate via Bamboo's *Plan permissions* — only users with "Release production" permission can trigger the Apply stage; ordinary developers cannot.

The combination means: plan branches show the diff but cannot apply; only the main-branch plan, post-merge, with a privileged reviewer's action, applies to production.

Decoupled S3 State Files

Per the AUTOM-09 §6 directory-per-env layout, each environment has its own `backend.tf` pointing at a separate S3 key:

```
# envs/dev/backend.tf
terraform {
  backend "s3" {
    bucket         = "my-org-terraform-state"
    key            = "dynatrace/dev.tfstate"
    region         = "us-east-1"
    encrypt        = true
    use_lockfile   = true   # S3-native locking (Terraform 1.11+); DynamoDB locking is deprecated
    kms_key_id     = "arn:aws:kms:us-east-1:123456789012:key/abcd-..."
  }
}
```

```
# envs/staging/backend.tf
terraform {
  backend "s3" {
    bucket         = "my-org-terraform-state"
    key            = "dynatrace/staging.tfstate"
    # ... same bucket / region / use_lockfile, different key
  }
}
```

```
# envs/production/backend.tf
terraform {
  backend "s3" {
    bucket         = "my-org-terraform-state"
    key            = "dynatrace/production.tfstate"
    # ... same bucket / region / use_lockfile, different key
  }
}
```

Decoupled here means: each environment has its own state file (its own key in S3, and its own `<key>.tflock` lock file next to it). The Bamboo plan for dev cannot accidentally affect production state because the working directory + `backend.tf` it picks up at `cd envs/<env>` selects a different state file.

One bucket vs many. A single bucket with separate keys per env is the simpler operational model and is fine for most shops. Separate buckets per env are warranted when production has different security posture (different KMS keys, different bucket policies, different access patterns). Both work; pick based on your security boundary.

IAM role on the Bamboo agent needs (minimum):

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": ["s3:GetObject", "s3:PutObject", "s3:DeleteObject"],
      "Resource": "arn:aws:s3:::my-org-terraform-state/dynatrace/*"
    },
    {
      "Effect": "Allow",
      "Action": ["s3:ListBucket"],
      "Resource": "arn:aws:s3:::my-org-terraform-state"
    },
    {
      "Effect": "Allow",
      "Action": ["kms:Encrypt", "kms:Decrypt", "kms:GenerateDataKey"],
      "Resource": "arn:aws:kms:us-east-1:123456789012:key/abcd-..."
    }
  ]
}
```

The `dynatrace/*` object statement also covers the `.tflock` lock files, which need `s3:GetObject`, `s3:PutObject` and `s3:DeleteObject`. Scope the resource ARN to the `dynatrace/` prefix so the role can't touch state for other systems sharing the bucket. For production-only roles (separate Bamboo agents for prod), narrow further to `dynatrace/production.tfstate`.

Bamboo Variables and Credentials

Bamboo has more variable types than the ones below (system and build-specific variables, for example); these are the scopes that matter for credentials:

Scope	Where set	When to use
Global variables	Bamboo Administration → Global Variables	Truly global values (e.g., a tenant URL shared across many plans). Visible to all plans.
Project variables	Project settings → Variables	Values shared by every plan in one Bamboo project; they override global variables of the same name.
Plan variables	Plan configuration → Variables; `variables:` block in YAML	Plan-specific values (the Dynatrace tokens for one tenant); they override global and project variables. Most credentials live here.
Plan-branch variables	Plan → Branch overrides	Per-branch overrides (e.g., a different tenant URL for a feature-branch plan testing against a sandbox tenant). Rarely needed.

Encrypted values for Specs YAML use the `BAMSCRT@0@0@<base64-encrypted-blob>` format. Generate them in the Bamboo UI: **Specs** → **Sensitive data encryption** → paste the token → **Encrypt** → copy the `BAMSCRT@0@0@…` value into the YAML.

Script tasks see variables with a `bamboo_` prefix. Bamboo *"variables are exported as bash shell variables. All full stops (periods) are converted to underscores."* — a plan variable `DYNATRACE_API_TOKEN` reaches a script as `$bamboo_DYNATRACE_API_TOKEN`. That is why each script in the plan above starts by exporting the `DYNATRACE_*` names the provider reads.

AWS authentication patterns (in preference order):

Approach	Trade-off
IAM role attached to Bamboo agent EC2 instance (recommended)	No credentials in Bamboo at all; AWS SDK auto-detects from instance metadata; rotation is AWS-side

Approach	Trade-off
IAM Identity Center (formerly SSO) federated role	Slightly more setup; works when Bamboo agents aren't on EC2
Static `AWS_ACCESS_KEY_ID` + `AWS_SECRET_ACCESS_KEY` as Bamboo encrypted plan variables	Works anywhere; requires manual rotation; encrypted-blob format protects them in the YAML

For the IAM-role pattern, the Terraform script doesn't need any AWS credentials in plan variables — the AWS provider auto-detects from EC2 instance metadata.

Dynatrace tokens are always encrypted variables. No exceptions — even in non-production environments, never store Dynatrace tokens as plaintext plan variables.

Manual Stages for Production Gating

The `manual: true` marker on a stage pauses the plan after the previous stage completes and waits for a user to explicitly start the manual stage. This is Bamboo's native gating mechanism — no plugin required.

```
stages:
  - Plan:
      jobs:
        - Plan
  - Apply:
      manual: true       # ← gate
      jobs:
        - Apply
```

Pair `manual: true` with stage permissions. The gate is only meaningful if not everyone can release it:

1. In the Bamboo UI under the plan's *Permissions* tab, define a "Release production" permission set with restricted membership.
2. Configure the manual stage to require that permission to start.
3. Result: developers can trigger plans and review output, but only authorized users can release the apply.

Auditing. Bamboo records who started each stage, with timestamp, in the plan's audit log. For SOX / SOC2 / regulated environments, this is the auditable record of who authorized each production apply. Retain it according to your control framework — Bamboo's default retention is configurable.

Operational note. When the Apply stage fails mid-execution (e.g., terraform apply hits an API rate limit and times out), Bamboo leaves the plan in a *failed* state. Recovery is a manual re-trigger of the Apply stage — Bamboo will run it against the same `tfplan` artifact subscribed from the original Plan stage. If state is partially-applied and you need to recover, follow AUTOM-09 §12 Operational Realities (stuck state lock and partial-apply recovery).

> **Sources:**
>
> - Bamboo YAML specs (Atlassian) — top-level structure (`version` , `plan` , `stages` , jobs, `script` tasks).
> - Bamboo Specs reference (Atlassian) — `manual: true` stage syntax, linked repositories referenced with `scope: global` (a `type: bitbucket` block *"Creates a local repository for the plan. This repository will not be shared among other plans."*), `variables` with `BAMSCRT@...` encrypted form, `branches` plan-branches configuration.
> - Bamboo variables (Atlassian) — *"For example, the variable bamboo.my.variable is $bamboo_my_variable in bash." "Project variables can override global variables with the same name."*
> - Bamboo Specs encryption (Atlassian) — *"From the top navigation bar, select Specs > Sensitive data encryption."*
> - Plan branches (Atlassian) — per-Bitbucket-branch child plans and lifecycle.
> - Terraform S3 backend (HashiCorp) — *"DynamoDB-based locking is deprecated and will be removed in a future minor version."* `use_lockfile` and the `.tflock` permissions: *"If use_lockfile is set, s3:GetObject, s3:PutObject, and s3:DeleteObject are required on the*

> *lock file"*.
>
> - **Derived:** the IAM policy snippet narrows the documented Terraform S3-backend permissions to the `dynatrace/` key prefix.

---

7. Azure DevOps Pipelines

Azure DevOps Pipelines is widely deployed in enterprise — particularly shops that already license the Microsoft ALM stack and want to keep CI/CD inside the same vendor ecosystem. This section recipes the YAML-pipeline pattern for Terraform-against-Dynatrace.

Critical distinction from the other platforms in this notebook: Azure DevOps separates *what runs* (defined in YAML, in the repo) from *when it can run* (approvals configured in the Azure DevOps web UI on Environments). The YAML author cannot bypass approvals — only the Environment owner can configure them. This is a deliberate separation-of-concerns property; it shows up as "approvals are not in YAML" in the docs.

Stack Assumed

Component	Choice
CI/CD	Azure DevOps Services (cloud) or Azure DevOps Server 2022+ (self-hosted)
Pipeline definition	YAML pipelines (recommended; UI-defined Classic pipelines are legacy)
Source	Any Git host — Azure Repos, GitHub, Bitbucket, GitLab — referenced via `resources.repositories`
Approval mechanism	**Environments** with **Approvals and checks** configured via UI (not in YAML) — see §7 *Environments and Approval Gates*
Credential storage	Variable Groups in the Library (optionally linked to Azure Key Vault for rotation)
Terraform task	`TerraformInstaller@1` comes from Microsoft DevLabs' Terraform extension (Visual Studio Marketplace) — install it in the organization first, or replace the task with a script step that downloads Terraform
Agent	Microsoft-hosted (default) or self-hosted; self-hosted required when the Terraform plan needs network access to a private Dynatrace tenant or on-prem state backend
AWS auth (if S3 backend)	Service connection (AzureRM type for Azure; AWS Toolkit for Azure DevOps extension for AWS) — preferred over static keys in variable groups

Pipeline YAML — Terraform with Combined Auth

The full plan covering plan + apply against one environment. Stages map to Plan (auto on every PR) and Apply (deployment job targeting an Environment that has manual-approval check configured).

```yaml
# azure-pipelines.yml — production
trigger:
  branches:
    include:
      - main
  paths:
    include:
      - envs/production/**
      - modules/**

pr:
  branches:
    include:
      - main
  paths:
    include:
      - envs/production/**
      - modules/**

variables:
  - group: dynatrace-production    # ← linked variable group; see §7 Variable Groups
  - name: TERRAFORM_VERSION
    value: latest

pool:
  vmImage: ubuntu-latest

stages:
  - stage: Plan
    displayName: Terraform Plan
    jobs:
      - job: PlanJob
        steps:
          - checkout: self
          - task: TerraformInstaller@1
            inputs:
              terraformVersion: $(TERRAFORM_VERSION)
          - script: |
              cd envs/production
              terraform init -input=false
              terraform plan -input=false -no-color -out=tfplan
              terraform show -no-color tfplan &gt; plan.txt
            displayName: terraform plan
            env:
              DYNATRACE_ENV_URL: $(DYNATRACE_ENV_URL)
              DYNATRACE_PLATFORM_TOKEN: $(DYNATRACE_PLATFORM_TOKEN)
              DYNATRACE_API_TOKEN: $(DYNATRACE_API_TOKEN)
              DYNATRACE_HTTP_OAUTH_PREFERENCE: "true"
          - publish: envs/production/tfplan
            artifact: tfplan
          - publish: envs/production/plan.txt
            artifact: plan-txt

  - stage: Apply
    displayName: Terraform Apply (production)
    dependsOn: Plan
    condition: and(succeeded(), eq(variables['Build.SourceBranch'], 'refs/heads/main'))
    jobs:
      - deployment: ApplyJob
        environment: dynatrace-production   # ← Environment with manual approval attached via UI
        strategy:
          runOnce:
            deploy:
              steps:
                - checkout: self
```

```
            - download: current
              artifact: tfplan
            - task: TerraformInstaller@1
              inputs:
                terraformVersion: $(TERRAFORM_VERSION)
            - script: |
                cd envs/production
                cp $(Pipeline.Workspace)/tfplan/tfplan .
                terraform init -input=false
                terraform apply -input=false -auto-approve tfplan
              displayName: terraform apply
              env:
                DYNATRACE_ENV_URL: $(DYNATRACE_ENV_URL)
                DYNATRACE_PLATFORM_TOKEN: $(DYNATRACE_PLATFORM_TOKEN)
                DYNATRACE_API_TOKEN: $(DYNATRACE_API_TOKEN)
                DYNATRACE_HTTP_OAUTH_PREFERENCE: "true"
```

Key elements:

- **`trigger:` / `pr:`** — main-branch push triggers full plan+apply; PR triggers plan-only (the Apply stage's `condition` skips it on non-main branches).
- **`stages:` (Plan, Apply)** — Plan runs on every trigger; Apply runs only on main-branch trigger AND only after the Environment's approval check passes.
- **`jobs.deployment` + `environment: dynatrace-production`** — the deployment job targets an Environment by name. When the Environment has *Approvals and checks → Approvals* configured via UI, the pipeline pauses on this job until an approver releases it.
- **`publish:` / `download:`** — the Plan stage publishes `tfplan` as a pipeline artifact; Apply downloads it to ensure the apply runs against the reviewed plan, not a re-plan.
- **`variables: - group:`** — references a variable group (defined in Azure DevOps Library) by name. Combined-auth tokens (`DYNATRACE_PLATFORM_TOKEN` , `DYNATRACE_API_TOKEN` , `DYNATRACE_ENV_URL` , `DYNATRACE_HTTP_OAUTH_PREFERENCE`) come from there.

Environments and Approval Gates (configured in UI, not YAML)

Per the Azure DevOps approvals docs: *"Approvals and other checks aren't defined in the yaml file. Users modifying the pipeline yaml file can't modify the checks performed before start of a stage. Administrators of resources manage checks using the web interface of Azure Pipelines."*

The implication: production safety is enforced **outside** the repo. Even a developer with write access to the YAML cannot remove the gate — only the Environment's administrator can.

To configure:

1. **Pipelines → Environments →** create an Environment named `dynatrace-production` (matches the `environment:` value in the YAML deployment job).
2. **Approvals and checks** tab → click **+** → **Approvals** → add user(s) or group(s) as Approvers, set Timeout, decide whether self-approval is permitted.
3. (Optional) Add additional checks — **Branch control** to restrict which branches can deploy here, **Business hours** to limit deploy windows, **Required template** to enforce a specific YAML template, **Invoke REST API** to gate on an external system (e.g., ServiceNow change-record validation).

The five check categories (run in this order at stage entry): Static checks (Branch control, Required template, Evaluate artifact) → Pre-check approvals → Dynamic checks (Approval, Invoke Azure Function, Invoke REST API, Business Hours, Query Azure Monitor) → Post-check approvals → Exclusive lock.

> **Bypass option:** an Environment administrator can **bypass** an approval for a stage that's waiting — useful for hotfixes but auditable (Azure DevOps records who bypassed). This is the operationally cleaner equivalent of "edit the YAML to

> skip the gate" — same effect, but logged and permission-scoped.

Variable Groups for Dynatrace Credentials

Variable Groups live in **Pipelines → Library**. Two flavors:

Flavor	Where the values live	When to use
Plain variable group	Encrypted in Azure DevOps storage	Smaller shops; tokens managed via Azure DevOps UI
Key Vault-linked variable group	Azure Key Vault; values refreshed at pipeline run	Enterprise; centralized secret rotation in Key Vault; audit trail of secret access

Variable mapping for combined auth:

Variable	Type	Required when
`DYNATRACE_ENV_URL`	Plaintext (URL — not secret)	Always
`DYNATRACE_PLATFORM_TOKEN`	Secret	Primary token is a Platform Token (`dt0s16` / `dt0s01`)
`DYNATRACE_API_TOKEN`	Secret	Primary is Platform Token AND you manage v1.88.0-excluded resources, OR primary itself is a classic `dt0c01` Token
`DYNATRACE_HTTP_OAUTH_PREFERENCE`	Plaintext (`"true"`)	When Platform Token is in play

Mark each secret variable's 🔒 **(Lock)** in the variable-group UI — Azure DevOps then redacts the value in logs and won't expose it in plan output.

Permission control: *Pipelines → Library → [your variable group] → Security →* restrict which pipelines can consume the group, and which users can edit it. Combine with Environment approvals so the credentials don't accidentally end up in a pipeline targeting the wrong tenant.

PR-Driven Plan-Only Workflow

The `pr:` trigger at the top of the YAML causes Azure DevOps to run the pipeline on PR open / update. By gating the Apply stage on the source branch (the `condition: eq(variables['Build.SourceBranch'], 'refs/heads/main')` clause), PR builds run Plan only — Apply is skipped automatically.

Best-practice composition:

1. **Branch policy on `main`** (configured in *Repos → Branches → main → Branch policies*) — require minimum reviewer count + require PR validation pipeline to pass. Reviewers see the Plan output as a build artifact / posted as a PR comment by a script step calling the Azure DevOps REST API to add a PR thread, or an actively-maintained marketplace task (check before adopting).
2. **PR triggers** the pipeline → Plan stage runs → publishes `plan.txt` as artifact + (optional) posts to PR.
3. **Reviewer approves** PR → merge to `main` → push triggers the pipeline → Plan re-runs → Apply stage queues at the Environment approval gate → approver releases → Apply runs against the new `tfplan` from the post-merge Plan.

The "Plan twice" property (once on PR, once on merge) is intentional — the merge commit can resolve conflicts and produce a different plan than the PR did. Treating the post-merge plan as the source of truth (and what the approver sees) closes the gap.

> **Sources:**
>
> - YAML schema reference (Microsoft Learn) — top-level structure (`trigger` , `pr` , `stages` , `jobs` , `jobs.deployment` , `variables` ,

> `pool`, `resources`).
>
> - Pipeline deployment approvals (Microsoft Learn) — *"Approvals and other checks aren't defined in the yaml file."* Five check categories (Static → Pre-check approvals → Dynamic → Post-check approvals → Exclusive lock). Manual Approval, Branch control, Business hours, Required template, Invoke REST API/Azure Function, Query Azure Monitor, Evaluate artifact.
> - Variable groups (Microsoft Learn) — Library variable groups, Key Vault linkage, per-group security/permissions.
> - **Community context:** Looking for Best Practices on using Terraform for Dynatrace automated through Azure DevOps (Dynatrace Guild) — May 2026 community thread where a guild member asks exactly this question; this section's structure mirrors the practitioner reply that emerged on the thread (two-layer repo model + state-key-derivation + plan-artifact reuse + manual-approval gates).

---

8. ArgoCD Integration

For Kubernetes-native GitOps, use ArgoCD to deliver the Dynatrace **Operator and DynaKube**.

> **Argo CD / Flux reconcile Kubernetes objects — not Monaco or Terraform projects.** Monaco YAML is not a Kubernetes manifest, and `monaco deploy` emits logs, not manifests, so it cannot be an Argo CD source or a Config Management Plugin `generate` command. Run Monaco/Terraform from CI (§3–§7) or from a Kubernetes Job, and use Argo CD / Flux for the Operator and DynaKube below.

ArgoCD for Dynatrace Operator (DynaKube)

Deploy and manage the Dynatrace Operator via ArgoCD:

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: dynatrace-operator
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/org/k8s-platform.git
    targetRevision: HEAD
    path: dynatrace
  destination:
    server: https://kubernetes.default.svc
    namespace: dynatrace
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
    syncOptions:
      - CreateNamespace=true
```

Repository structure for Dynatrace Operator:

```
dynatrace/
├── kustomization.yaml
├── namespace.yaml
├── operator.yaml          # Dynatrace Operator deployment
├── dynakube.yaml          # DynaKube custom resource
└── secrets/
    └── dynakube-secret.yaml  # External Secrets reference
```

External Secrets for Token Management

Use External Secrets Operator to sync tokens from secret stores:

```
apiVersion: external-secrets.io/v1
kind: ExternalSecret
metadata:
  name: dynakube-tokens
  namespace: dynatrace
spec:
  refreshInterval: 1h
  secretStoreRef:
    name: aws-secrets-manager
    kind: ClusterSecretStore
  target:
    name: dynakube
    creationPolicy: Owner
  data:
    - secretKey: apiToken
      remoteRef:
        key: dynatrace/api-token
    - secretKey: dataIngestToken
      remoteRef:
        key: dynatrace/data-ingest-token
```

External Secrets Operator stopped serving `v1beta1` in v0.17.0 — use `external-secrets.io/v1`. (The `argocd-cm` `configManagementPlugins` mechanism shown in earlier versions of this notebook was removed in Argo CD 2.8, and a CMP cannot run Monaco anyway — see the note at the top of this section.)

---

9. FluxCD Integration

FluxCD provides an alternative GitOps approach with a pull-based reconciliation model.

FluxCD vs ArgoCD

Feature	FluxCD	ArgoCD
Architecture	Controller-based	Server + UI
UI	Minimal (Weave GitOps)	Built-in web UI
Multi-tenancy	Namespace-based	Project-based
Helm Support	Native HelmRelease	Application CRD
Image Automation	Built-in	Requires Argo CD Image Updater

FluxCD for Dynatrace Operator

GitRepository source:

```yaml
apiVersion: source.toolkit.fluxcd.io/v1
kind: GitRepository
metadata:
  name: dynatrace-config
  namespace: flux-system
spec:
  interval: 1m
  url: https://github.com/org/k8s-platform
  ref:
    branch: main
  secretRef:
    name: github-token
```

Kustomization for Dynatrace:

```yaml
apiVersion: kustomize.toolkit.fluxcd.io/v1
kind: Kustomization
metadata:
  name: dynatrace
  namespace: flux-system
spec:
  interval: 10m
  targetNamespace: dynatrace
  sourceRef:
    kind: GitRepository
    name: dynatrace-config
  path: ./dynatrace
  prune: true
  healthChecks:
    - apiVersion: apps/v1
      kind: Deployment
      name: dynatrace-operator
      namespace: dynatrace
```

FluxCD HelmRelease for Dynatrace Operator

Deploy via Helm with FluxCD:

```yaml
apiVersion: source.toolkit.fluxcd.io/v1
kind: HelmRepository
metadata:
  name: dynatrace
  namespace: flux-system
spec:
  interval: 1h
  url: https://raw.githubusercontent.com/Dynatrace/dynatrace-operator/main/config/helm/repos/stable

---
apiVersion: helm.toolkit.fluxcd.io/v2
kind: HelmRelease
metadata:
  name: dynatrace-operator
  namespace: dynatrace
spec:
  interval: 5m
  chart:
    spec:
      chart: dynatrace-operator
      version: "1.10.4"   # exact pin; bump in a reviewed commit (see §10)
      sourceRef:
        kind: HelmRepository
        name: dynatrace
        namespace: flux-system
  values:
    installCRD: true
```

Pin the chart version exactly. A range such as `">=1.0.0"` resolves to the newest chart in the repository index — which lists `1.11.0` first on 10/02/2026, the release that removes `v1beta4` — so every reconcile would upgrade the Operator unattended, the same failure §10 warns about for the kustomize manifest.

Flux v2.7.0 removed the `v1beta1` and `v2beta1` APIs from its CRDs, so `HelmRelease` must be `helm.toolkit.fluxcd.io/v2` and `HelmRepository` `source.toolkit.fluxcd.io/v1`.

SOPS for Secret Management with FluxCD

Use SOPS to encrypt secrets in Git:

```yaml
apiVersion: kustomize.toolkit.fluxcd.io/v1
kind: Kustomization
metadata:
  name: dynatrace-secrets
  namespace: flux-system
spec:
  interval: 10m
  sourceRef:
    kind: GitRepository
    name: dynatrace-config
  path: ./dynatrace/secrets
  prune: true
  decryption:
    provider: sops
    secretRef:
      name: sops-age
```

---

10. Dynatrace Operator GitOps Patterns

When deploying the Dynatrace Operator via GitOps, follow these patterns for production environments.

> **Important:** Use `apiVersion: dynatrace.com/v1beta6` for new DynaKubes (`v1beta5` is still served, but flagged deprecated from Operator 1.10.0). Operator 1.9.0 removed `v1beta3` from the CRD and deprecated `v1beta4`, Operator 1.10.0 (July 15, 2026) stopped serving `v1beta4`, and Operator **1.11.0** (released 10/01/2026) removes it from the CRD — *"Applying DynaKube resources that still use v1beta4 will fail."* Audit committed DynaKube manifests for stale API versions before an Operator upgrade rolls through the pipeline.
>
> **Sources:** Operator 1.11.0 release notes (DT docs) — *"The v1beta4 version has been removed from the DynaKube CRD."* `served` / `deprecated` per version read from the DynaKube CRD in each release's `kubernetes.yaml` (Operator releases (Dynatrace GitHub)), 10/02/2026.

Multi-Cluster Deployment Pattern

For organizations with multiple Kubernetes clusters:

```
platform-gitops/
├── clusters/
│   ├── production-east/
│   │   └── dynatrace/
│   │       ├── kustomization.yaml
│   │       └── dynakube-patch.yaml     # Cluster-specific overrides
│   ├── production-west/
│   │   └── dynatrace/
│   │       ├── kustomization.yaml
│   │       └── dynakube-patch.yaml
│   └── staging/
│       └── dynatrace/
│           ├── kustomization.yaml
│           └── dynakube-patch.yaml
└── base/
    └── dynatrace/
        ├── kustomization.yaml
        ├── namespace.yaml
        ├── operator.yaml
        └── dynakube.yaml               # Base DynaKube config
```

Base kustomization.yaml:

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
  - namespace.yaml
  - https://github.com/Dynatrace/dynatrace-operator/releases/download/v1.10.4/kubernetes.yaml
  - dynakube.yaml
```

> **Pin the Operator manifest.** A `releases/latest/download/…` URL resolves to whichever Operator release is newest at build time — on 10/02/2026 that is **1.11.0**, the release that removes `v1beta4`. Every sync would then roll out an Operator upgrade nobody chose. Pin the version you validated (`v1.10.4` here) and move it in a reviewed commit, after reading the release notes for every version you cross.

Cluster overlay (production-east):

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
  - ../../base/dynatrace
patches:
  - path: dynakube-patch.yaml
```

Cluster-specific patch:

```
apiVersion: dynatrace.com/v1beta6
kind: DynaKube
metadata:
  name: dynakube
  namespace: dynatrace
spec:
  oneAgent:
    hostGroup: production-east   # replaces the obsolete --set-host-group argument
```

Multi-Tenant Observability Pattern

For clusters serving multiple teams with different Dynatrace tenants — **only when each team has its own node pool**. OneAgent host monitoring is one agent per node: the Operator rejects a second DynaKube whose OneAgent would land on nodes another DynaKube already covers, and `namespaceSelector` does not separate them, because it scopes code-module injection, not where the host agent runs. Give each DynaKube a disjoint `nodeSelector`:

```
# Team A - uses tenant-a.live.dynatrace.com, on the team-a node pool
apiVersion: dynatrace.com/v1beta6
kind: DynaKube
metadata:
  name: team-a-dynakube
  namespace: dynatrace
spec:
  apiUrl: https://tenant-a.live.dynatrace.com/api
  tokens: team-a-tokens
  oneAgent:
    cloudNativeFullStack:
      nodeSelector:
        dynatrace-tenant: team-a        # node label — OneAgent runs only on these nodes
      namespaceSelector:
        matchLabels:
          dynatrace-tenant: team-a      # namespace label — which pods get injected

---
# Team B - uses tenant-b.live.dynatrace.com, on the team-b node pool
apiVersion: dynatrace.com/v1beta6
kind: DynaKube
metadata:
  name: team-b-dynakube
  namespace: dynatrace
spec:
  apiUrl: https://tenant-b.live.dynatrace.com/api
  tokens: team-b-tokens
  oneAgent:
    cloudNativeFullStack:
      nodeSelector:
        dynatrace-tenant: team-b
      namespaceSelector:
        matchLabels:
          dynatrace-tenant: team-b
```

Keep each team's workloads on its own node pool as well (node affinity or taints), so an injected pod always runs on a node monitored by the same tenant's OneAgent. If teams have to share nodes, this pattern does not apply — one DynaKube must own host monitoring on those nodes.

> **Sources:**
>
> - Migrate to a single DynaKube (DT docs) — *"Dynatrace Operator rejects a DynaKube that conflicts with another one over OneAgent node assignment, namespace injection, or telemetry ingest service names."*
> - DynaKube validation, Operator v1.11.0 (Dynatrace GitHub) — *"Only one Agent per node is supported. Use a nodeSelector to avoid this conflict."*
> - DynaKube parameters (DT docs) — `nodeSelector`: *"Specify the node selector that controls on which nodes OneAgent will be deployed."* `hostGroup`: *"This method is preferred over the now obsolete --set-host-group argument."*

GitOps Health Checks

Keep the DynaKube's changing status out of Argo CD's drift view, and let Flux gate a sync on the Operator and OneAgent workloads:

ArgoCD — ignore DynaKube status drift (an `ignoreDifferences` block, not a health check):

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: dynatrace
spec:
  # ... source config ...
  ignoreDifferences:
    - group: dynatrace.com
      kind: DynaKube
      jsonPointers:
        - /status  # Ignore status field changes
```

FluxCD health check:

```
apiVersion: kustomize.toolkit.fluxcd.io/v1
kind: Kustomization
metadata:
  name: dynatrace
spec:
  healthChecks:
    - apiVersion: apps/v1
      kind: Deployment
      name: dynatrace-operator
      namespace: dynatrace
    - apiVersion: apps/v1
      kind: DaemonSet
      name: dynakube-oneagent
      namespace: dynatrace
```

Sealed Secrets Alternative

For Kubernetes-native secret encryption:

```
apiVersion: bitnami.com/v1alpha1
kind: SealedSecret
metadata:
  name: dynakube
  namespace: dynatrace
spec:
  encryptedData:
    apiToken: AgBy8BYo...encrypted...
    dataIngestToken: AgCtr4s2...encrypted...
```

Generate sealed secrets:

```
# Seal the secret (use your actual tokens from Dynatrace)
kubectl create secret generic dynakube \
  --from-literal=apiToken=&lt;your-api-token&gt; \
  --from-literal=dataIngestToken=&lt;your-data-ingest-token&gt; \
  --dry-run=client -o yaml | \
  kubeseal --format yaml &gt; sealed-dynakube.yaml
```

---

11. Ephemeral Platform Token Minting Pattern

Applies across every CI platform in §§3–§7 — it's an architectural pattern, not a platform feature. The goal: **eliminate long-lived Dynatrace credentials from CI/CD runners**. The only credential that persists is the OAuth client that bootstraps the chain; the actual Dynatrace tokens used by `terraform apply` are minted per-job, used for the job's lifetime (typically ≤1 hour), and deleted on completion. This is the cloud-IAM-shop equivalent of AWS STS `AssumeRole` issuing short-lived session credentials, or GCP workload-identity federation issuing OIDC-bound access tokens.

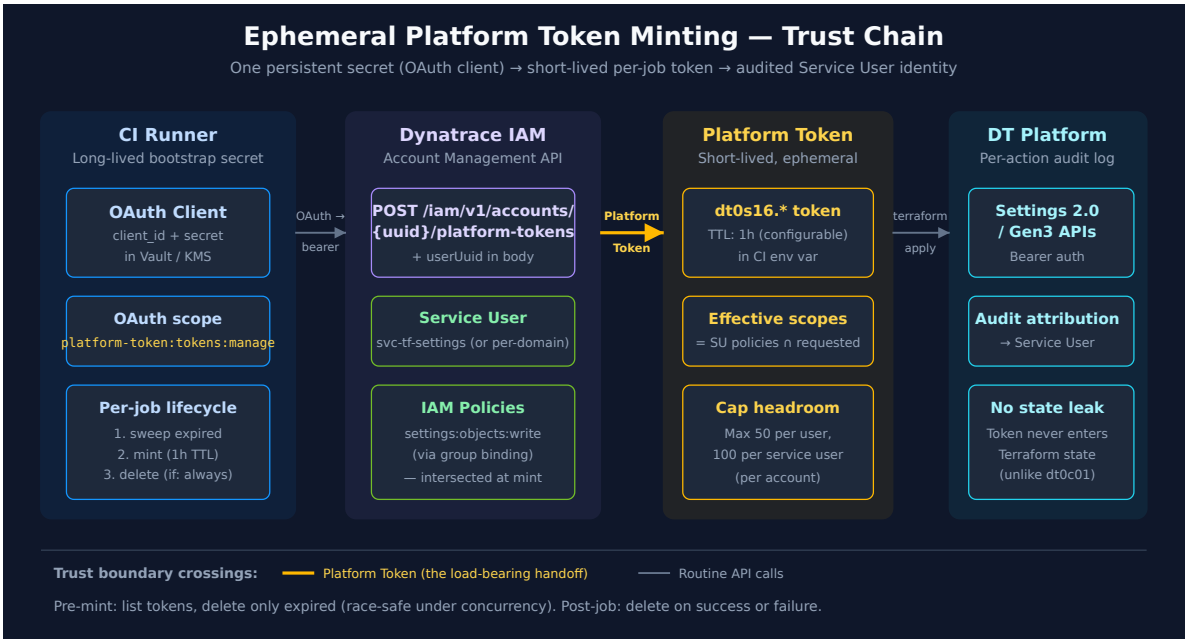
Why bother with the extra moving parts? Three reasons:

1. **No long-lived token in CI runners.** A leaked runner image, a misconfigured log, a curious insider — none of them gets a usable Dynatrace credential. The OAuth client_id+secret is the single persistent secret; it lives in one place (a real secret manager) and is consumed only at job-start.
2. **Per-job audit attribution.** Every `terraform apply` call is attributable to the Service User on whose behalf the Platform Token was minted (`svc-tf-settings` , `svc-tf-iam` , etc.) — not to a generic shared admin identity or to an OAuth client identifier.
3. **No Terraform state leakage.** Platform Tokens never enter Terraform state. Contrast `dynatrace_api_token` , where the minted classic API Token persists plain-text in state (see AUTOM-04 §3 *Operational Safety — State File Leakage*).

Prerequisite — Platform Tokens cannot be minted by Terraform. The Dynatrace Terraform provider has no `dynatrace_platform_token` resource (verified against provider source 2026-05-22 — see AUTOM-04 §3). This section operates at the layer below Terraform: shell or pipeline orchestration that obtains a Platform Token before `terraform apply` runs, and disposes of it afterward.

> **Early Access — workload identity federation.** Dynatrace's Workload identity federation (DT docs) page says *"Workload identity federation is an Early Access feature."* Where it is enabled, a workload's own OIDC token is resolved to a Service User, *"so you never store a Dynatrace client secret or platform token alongside your workload"*, and the page names *"a GitHub Actions job"* among suitable workloads. That would remove the persistent OAuth client this pattern starts from. Until WIF is available in your account, the OAuth-client bootstrap below remains the working path.

11.1 Conceptual Model — Four-Layer Trust Chain



The four layers, reading left-to-right in the diagram:

Layer 1 — CI Runner with OAuth Client. A persistent OAuth client_id+secret pair lives in a secret manager (Vault, AWS Secrets Manager, Azure Key Vault, GCP Secret Manager). The OAuth client carries **exactly one OAuth scope:** `platform-token:tokens:manage` — the scope the mint endpoint requires. It grants no user/group/policy changes and no Settings/Synthetics/SLO operations, but it is not minimal: the IAM reference describes `platform-token:tokens:manage` as *"admin-level access to list, update, and delete any platform token within an account regardless of owner"*, so treat this client's secret as an account-level credential.

Layer 2 — Dynatrace IAM, holding one or more Service Users. Each Service User (provisioned per AUTOM-95 §4) carries the actual IAM policies for the work the pipeline performs — `settings:objects:write` for the settings pipeline, `synthetic:monitors:write` for synthetic monitors managed through the platform, etc. (The Terraform provider's synthetic resources still need a classic API token — see §11.5.) The Service User is what the per-job Platform Token will be *on behalf of*.

Layer 3 — Per-Job Platform Token. At job start, CI calls `POST /iam/v1/accounts/{accountUuid}/platform-tokens` with the OAuth bearer in the `Authorization` header and the Service User's UUID (`dynatrace_iam_service_user.id`) as the `userUuid` body field. Dynatrace returns a `dt0s16.*` Platform Token. **Effective permissions = Service User's IAM policies ∩ scopes requested in the mint body** (the intersection model documented at Platform tokens (DT docs)). Always request narrower scopes than the Service User holds — gives per-job blast-radius control on top of identity-level least-privilege.

Layer 4 — Dynatrace platform APIs. `terraform apply` (or any other API client in the job) uses the Platform Token via `Authorization: Bearer`. Audit log entries are attributed to the Service User, not the OAuth client.

The load-bearing handoff is the amber arrow — Platform Token returned from the IAM API to the CI runner. That's where the token crosses a trust boundary as a short-lived secret. Everything else in the diagram is routine API surface.

> **Sources:** POST /iam/v1/accounts/{accountUuid}/platform-tokens (DT docs), Platform tokens (DT docs) for the intersection model. IAM policy statements reference (DT docs) — *"Grants admin-level access to list, update, and delete any platform token within an account regardless of owner."* For synthetics, `synthetic:monitors:write` : *"Grants permission to create synthetic monitors"*. The mint endpoint page states the scope: *"you need an OAuth client with the platform-token:tokens:manage scope assigned"* (re-read 09/28/2026; the endpoint's `x-token-`

> `scopes` in the Account Management API spec at `api.dynatrace.com/spec` agrees). **Derived:** the four-layer composition is a synthesis — Dynatrace docs describe each layer separately but do not endorse this composition as a named pattern.

11.2 Prerequisites — OAuth Client + Service Users + IAM Policies

Before any CI job can mint a token, the IAM scaffolding must exist:

1. A dedicated OAuth client for minting only. Create an OAuth client in `myaccount.dynatrace.com` with **exactly one scope:** `platform-token:tokens:manage`. Do not give this OAuth client other scopes — every additional scope expands its blast radius if compromised. Store the `client_id` + `secret` in your secret manager (Vault, AWS Secrets Manager, Azure Key Vault, GCP Secret Manager).

Note on scope source-of-truth (corrected 09/28/2026): the Platform Token mint endpoint docs page and the endpoint's `x-token-scopes` in the Account Management API spec at `api.dynatrace.com/spec` now agree on `platform-token:tokens:manage` — the same scope the list, delete, expiration and status endpoints used in §11.3 declare. Earlier revisions of this section used `platform-token:tokens:write` and noted `account-idm-write`; neither source lists either one for this endpoint any more. The IAM reference defines `platform-token:tokens:write` as access to *the caller's own* platform tokens only, which does not cover minting for a Service User. If your account rejects `platform-token:tokens:manage`, re-read the spec before reaching for a broader scope.

2. Domain-scoped Service Users via Terraform. Create Service Users by domain, not one mega-user. The pattern in AUTOM-95 §4 (`svc-tf-settings` , `svc-tf-iam` , `svc-tf-synth` , `svc-tf-slo` , optionally `svc-tf-grail` and `svc-tf-workflows`). Each Service User holds **only the IAM policies appropriate to its domain** via group bindings (AUTOM-95 §10). Compromise of a per-job Platform Token gives the attacker exactly that domain's permissions, no more.

3. IAM policies + group bindings already applied. The Service User must be in groups bound to policies that grant the action your `terraform apply` will perform. If the mint succeeds but the apply gets HTTP 403, the most likely cause is that the requested scope was correct but the Service User's IAM policies don't allow the action — see the intersection-model note in §11.1.

4. The Service User's UUID. This is the value `dynatrace_iam_service_user.svc_tf_settings.id` (or equivalent) exports. Make it available to the pipeline as a non-secret env var (`SVC_USER_ID`); it's identifying but not secret.

11.3 Worked Recipe — Pre-Mint Sweep, Mint, Use, Delete

The lifecycle has four steps. The pre-mint sweep is the part most home-grown implementations skip — it's what keeps the pattern safe under concurrent CI jobs.

The script relies on these documented response shapes (abbreviated):

```
GET  /iam/v1/accounts/{accountUuid}/platform-tokens?page=1&amp;size=100
     → { "pageSize": …, "pageNumber": 1, "total": …,
         "results": [ { "tokenId": "…", "userUuid": "…", "expirationDate": "2026-12-01T00:00:00Z", … } ] }
POST /iam/v1/accounts/{accountUuid}/platform-tokens
     → { "name": "…", "tokenId": "…", "token": "&lt;secret&gt;" }
DELETE /iam/v1/accounts/{accountUuid}/platform-tokens/{platformTokenId}   (platformTokenId = tokenId)
```

```
#!/usr/bin/env bash
# Run this script at the start of every terraform-apply CI job.
# Expected env vars (set by the CI platform, sourced from secret manager):
#   DT_ACCOUNT_UUID         — the account UUID
#   DT_OAUTH_CLIENT_ID      — OAuth client created in step 11.2 (1)
#   DT_OAUTH_CLIENT_SECRET  — its secret
#   SVC_USER_ID             — the Service User UUID for this domain
#   REQUESTED_SCOPES        — e.g. '["settings:objects:write"]'
#   TOKEN_RESOURCE          — e.g. '["urn:dtenvironment:&lt;environment-id&gt;"]' or
'["urn:dtaccount:&lt;account-uuid&gt;"]'
#   JOB_ID                  — unique job identifier for token name
set -euo pipefail

API="https://api.dynatrace.com"
TOKENS="${API}/iam/v1/accounts/${DT_ACCOUNT_UUID}/platform-tokens"

# ----- Step 1: Exchange OAuth client_id+secret for a bearer token -----
# The bearer is short-lived (the documented example response shows "expires_in": 300),
# so cleanup fetches a fresh one instead of reusing this one after a long apply.
get_bearer() {
  curl -sS --fail-with-body -X POST "https://sso.dynatrace.com/sso/oauth2/token" \
    -H "Content-Type: application/x-www-form-urlencoded" \
    --data-urlencode "grant_type=client_credentials" \
    --data-urlencode "client_id=${DT_OAUTH_CLIENT_ID}" \
    --data-urlencode "client_secret=${DT_OAUTH_CLIENT_SECRET}" \
    --data-urlencode "scope=platform-token:tokens:manage" \
    --data-urlencode "resource=urn:dtaccount:${DT_ACCOUNT_UUID}" |
    jq -er '.access_token'
}
OAUTH_BEARER=$(get_bearer)

# ----- Step 2: Pre-mint sweep — delete ONLY expired tokens for this Service User -----
# Race-safe under concurrency: active tokens (expirationDate in the future) and
# never-expiring tokens are NEVER touched, so a parallel job's in-flight token
# cannot be revoked. The list endpoint has no per-user filter (only searchTerm,
# status, page and size), so filter on userUuid client-side, page by page.
NOW_EPOCH=$(date -u +%s)
EXPIRED_IDS=""
PAGE=1   # the documented example response starts at pageNumber 1
while :; do
  BODY=$(curl -sS --fail-with-body -H "Authorization: Bearer ${OAUTH_BEARER}" \
    "${TOKENS}?page=${PAGE}&amp;size=100")
  EXPIRED_IDS+=$(echo "${BODY}" | jq -r --arg u "${SVC_USER_ID}" --argjson now "${NOW_EPOCH}" '
      .results[]
      | select(.userUuid == $u and .expirationDate != null)
      # fromdateiso8601 rejects fractional seconds, so strip them first
      | select((.expirationDate | sub("\\.[0-9]+"; "") | fromdateiso8601) &lt; $now)
      | .tokenId')$'\n'
  COUNT=$(echo "${BODY}" | jq '.results | length')
  TOTAL=$(echo "${BODY}" | jq '.total')
  if [ "${COUNT}" -eq 0 ] || [ $((PAGE * 100)) -ge "${TOTAL}" ]; then break; fi
  PAGE=$((PAGE + 1))
done
for expired_id in ${EXPIRED_IDS}; do
  curl -sS --fail-with-body -X DELETE -H "Authorization: Bearer ${OAUTH_BEARER}" \
    "${TOKENS}/${expired_id}" &gt;/dev/null \
    || echo "warning: could not delete expired token ${expired_id}" &gt;&amp;2
done

# ----- Step 3: Mint a per-job Platform Token with 1h TTL -----
EXPIRY=$(date -u -d '+1 hour' +%Y-%m-%dT%H:%M:%SZ 2&gt;/dev/null \
  || date -u -v +1H +%Y-%m-%dT%H:%M:%SZ)   # GNU date vs BSD date fallback

# name, scope, resource, tags, expirationDate and userUuid are all marked
# Required in the endpoint's request-body model.
```

```
MINT_RESPONSE=$(curl -sS --fail-with-body -X POST \
  -H "Authorization: Bearer ${OAUTH_BEARER}" \
  -H "Content-Type: application/json" \
  -d "$(jq -n \
       --arg name "ci-job-${JOB_ID}" \
       --arg uuid "${SVC_USER_ID}" \
       --arg exp "${EXPIRY}" \
       --argjson scope "${REQUESTED_SCOPES}" \
       --argjson resource "${TOKEN_RESOURCE}" \
       '{name: $name, scope: $scope, resource: $resource, tags: ["ci-ephemeral"],
         expirationDate: $exp, userUuid: $uuid}')" \
  "${TOKENS}") || { echo "mint request failed" &gt;&amp;2; exit 1; }

# The response carries the ID as tokenId. A missing ID would make the cleanup
# below delete nothing, silently — so stop here instead.
DT_TOKEN_ID=$(echo "${MINT_RESPONSE}" | jq -er '.tokenId') \
  || { echo "mint response had no tokenId" &gt;&amp;2; exit 1; }

# ----- Cleanup: registered as soon as a token exists -----
cleanup_token() {
  local bearer
  # Fresh bearer: the step-1 bearer may have expired during a long apply
  if ! bearer=$(get_bearer); then
    echo "cleanup: no bearer; token ${DT_TOKEN_ID} expires at ${EXPIRY}" &gt;&amp;2
    return
  fi
  curl -sS --fail-with-body -X DELETE -H "Authorization: Bearer ${bearer}" \
    "${TOKENS}/${DT_TOKEN_ID}" &gt;/dev/null \
    || echo "cleanup: delete of ${DT_TOKEN_ID} failed; it expires at ${EXPIRY} and the next sweep removes
it" &gt;&amp;2
}
trap 'cleanup_token' EXIT     # cleanup on success OR failure

# A platform token (dt0s16) goes in the PLATFORM token variable — in DT_API_TOKEN the
# provider would send it as "Api-Token" and get a 401
DYNATRACE_PLATFORM_TOKEN=$(echo "${MINT_RESPONSE}" | jq -er '.token')
export DYNATRACE_PLATFORM_TOKEN
export DYNATRACE_HTTP_OAUTH_PREFERENCE=true

# ----- Step 4: Run the work (terraform apply, etc.) -----
terraform apply -auto-approve   # the actual work
```

Key points the script encodes:

- **Pre-mint sweep deletes only expired tokens.** Active tokens (`expirationDate > now`) and tokens with no expiration are never touched, so a parallel CI job's in-flight token cannot be revoked mid-apply. In community practice, the naive *delete-oldest-N* approach is avoided: under concurrent jobs it can revoke another job's in-flight token.
- **The sweep filters and pages client-side.** The list endpoint takes only `searchTerm` , `status` , `page` and `size` — there is no user filter — and returns `{pageSize, pageNumber, total, results}` . The response model lists `userUuid` on each entry, but the page's example body shows an `owner` field instead; run one list call against your account and confirm which field carries the Service User's UUID before trusting the sweep. Note also that the sweep reads tokens for **every** user in the account, which is what the account-wide `platform-token:tokens:manage` scope allows.
- **Token IDs are `tokenId` .** Both the list entries and the mint response use `tokenId` (there is no `id`), and the DELETE path's `{platformTokenId}` takes that value. The script stops if the mint response has no `tokenId` , rather than registering a cleanup that would delete nothing.
- **All six mint-body fields are sent.** `resource` narrows the token to an environment (`urn:dtenvironment:…`) or the account (`urn:dtaccount:…`); the docs' example uses the account URN. The OAuth token request also passes `resource=urn:dtaccount:<uuid>` , as the OAuth client docs show.
- **Short TTL (1h).** Orphaned tokens from crashed CI runners expire into eligibility for the next job's sweep — self-healing capacity.

- `trap ... EXIT` **cleanup with a fresh bearer.** The trap is registered as soon as a token exists and deletes it on success, failure or signal interruption. It requests a new OAuth bearer first, because the step-1 bearer may have expired during the apply, and it logs a failed delete instead of hiding it — the token then expires on its own and the next sweep removes it.
- `REQUESTED_SCOPES` **narrower than Service User policies.** Take the intersection-model behaviour seriously — pass the narrowest scope set the job actually needs in the mint body, not the maximum the Service User could grant.
- **Error code for cap-exceeded is undocumented.** The mint endpoint docs document only the 200 response. If the cap is exceeded, the actual HTTP code (likely 400/403/429) is not stated at source. Treat any non-200 from mint as "sweep + retry once" — empirically verify the code against your tenant before adding finer error handling.

> **Sources:**
>
> - POST /iam/v1/accounts/{accountUuid}/platform-tokens (DT docs) — request-body model marks `name`, `scope`, `resource`, `tags`, `expirationDate` and `userUuid` as Required; response `{ "name", "tokenId", "token" }`.
> - GET all platform tokens (DT docs) — *"Only platform tokens that match values of tokenId, name, createdBy or scope will be returned."* Response `{pageSize, pageNumber, total, results}`.
> - DELETE a platform token (DT docs) — `DELETE …/platform-tokens/{platformTokenId}`.
> - OAuth clients (DT docs) — token request parameters including `resource` `urn:dtaccount:{your-account-UUID}`; example response `"expires_in": 300`.
> - Platform tokens (DT docs) — *"Platform tokens can be set to expire after a period of time or never expire."* *"A platform token can be further reduced in scope to only target one or many environments within the account the token is being issued against."*
> - **Derived:** the pre-mint sweep + short-TTL + `trap EXIT` lifecycle combines the mint/list/delete endpoints; Dynatrace docs do not describe it as a named pattern.

11.4 Capacity Planning Under the Per-User Cap

Verified at source (re-read 09/18/2026): *"A maximum of 50 platform tokens can be generated by a user per account, or 100 for a service user."* (Platform tokens (DT docs)). The cap was 10 per user when this section was first written; it is **per user, per account** — i.e., per `userUuid`, which means **per Service User** (100) in the on-behalf-of model. Multiple OAuth clients minting for the same Service User all count against that user's cap.

The arithmetic: with **N Service Users × 100 tokens** = N×100 capacity. A typical Terraform-driven CI shop with the AUTOM-95 §4 starter set (`svc-tf-settings`, `svc-tf-iam`, `svc-tf-synth`, `svc-tf-slo`) gets 400 token-slots of capacity. With short TTLs and the pre-mint sweep, steady-state utilization should rarely exceed concurrent-job count per domain — at the current cap, capacity is rarely the reason to add Service Users; attribution is.

Mitigation patterns by concurrency tier: the tier boundaries follow from the 100-token cap; the recommendations are community practice rather than Dynatrace guidance — tune them to your own pipeline.

Concurrent applies per domain	Recommendation	Why
1–30	Pattern works as-is with domain-scoped Service Users	100-cap is far from binding; sweep handles orphans
31–99	Keep the pre-mint sweep + monitor cap headroom in a dashboard	Approaching cap; orphan accumulation becomes load-bearing
≥100	**Pattern requires Service User fleet** — not a drop-in	One Service User cannot serve sustained ≥100-concurrent without burst failures

Fleet extension when a single domain saturates a single Service User:

```
# Three replicas, same IAM policies, same group memberships — purely capacity
resource "dynatrace_iam_service_user" "svc_tf_settings" {
  for_each    = toset(["01", "02", "03"])
  name        = "svc-tf-settings-${each.key}"
  description = "Settings 2.0 Terraform CI worker ${each.key}"
  groups      = [dynatrace_iam_group.monitoring_settings_writers.id]
}
```

Then in the CI runner, hash `JOB_ID mod N` → pick the Service User UUID to use:

```
# Round-robin selection across the Service User fleet
SVC_USER_IDS=(
  "${SVC_TF_SETTINGS_01_ID}"
  "${SVC_TF_SETTINGS_02_ID}"
  "${SVC_TF_SETTINGS_03_ID}"
)
INDEX=$(( $(echo "${JOB_ID}" | cksum | awk '{print $1}') % ${#SVC_USER_IDS[@]} ))
SVC_USER_ID="${SVC_USER_IDS[${INDEX}]}"
```

Add fleet replicas reactively, not preemptively. In community practice, provision the second replica when dashboards show sustained pressure (cap-utilization >70% during business hours). Provisioning a 5-node fleet up front for a domain that turns out to handle 2 concurrent applies is overengineering and dilutes audit attribution.

Critical reminder — the cap is on the Service User, not the OAuth client. Shops with multiple CI systems (one OAuth client per platform — GitHub Actions + Azure DevOps + Bamboo, etc.) all minting for the same Service User share its 100-token cap. Capacity planning must aggregate across all minting sources, not per OAuth client.

> **Sources:** Platform tokens (DT docs) for the cap verbatim. **Derived:** the tier boundaries in the concurrency table follow from the 100-per-service-user cap. Empirically verify the exact HTTP error code for cap-exceeded against your tenant before tuning automated retry logic.

11.5 When This Pattern Is Justified

In community practice, the pattern earns its operational overhead in the cases below and not in the ones after them — weigh both lists against your own audit and blast-radius requirements.

Use it for:

- **High-blast-radius applies** — production IAM changes (AUTOM-95), security configurations, audit-relevant Settings 2.0 changes. The per-job attribution + no-state-leakage justifies the operational complexity.
- **High-frequency CI** — roughly ten or more applies per day per pipeline (a community rule of thumb — verify the break-even against your own overhead). The compound benefit of "no long-lived token in any runner" compounds across runs.
- **Multi-team shared platform** — multiple teams contributing to Dynatrace config via the same pipeline. Per-domain Service Users give clean audit attribution that a shared admin token cannot.
- **Regulatory environments** — SOX, SOC2, FedRAMP audits care about "every administrative action attributable to a named identity". This pattern delivers that natively; long-lived shared admin tokens do not.

Don't use it for:

- **Single-team, low-frequency dev applies** — a few applies per week against a non-production tenant. The simpler pattern (long-lived Platform Token on a Service User, stored in your CI platform's secret store) is fine. The composed pattern's operational overhead is overkill for the blast radius.
- **Synthetic monitors / classic SLOs / `dynatrace_api_token`** — these resources cannot use Platform Tokens at all (AUTOM-04 §3 Decision matrix). They require classic API Tokens (`dt0c01.*`), which are minted by the `dynatrace_api_token` Terraform resource with the state-leakage trade-off documented in AUTOM-04 §3 *Operational Safety*.

- **Bootstrap / pre-IAM scenarios** — the very first time you stand up Dynatrace automation, you don't yet have the Service Users or IAM policies in place. Run that bootstrap pipeline with an admin's Platform Token, then transition subsequent runs to this pattern. Don't try to apply this pattern to the pipeline that creates the Service Users it depends on (chicken-and-egg).

Decision shortcut — three signals that flip the recommendation to "adopt":

Signal	Why
Auditor asks "who applied this configuration?"	Long-lived shared admin tokens cannot answer cleanly; this pattern can
Token leak post-mortem	Once it has happened to your shop, the case for ephemeral tokens makes itself
Cross-team Dynatrace pipeline	Per-domain attribution becomes a governance requirement, not a nicety

Cross-references:

- AUTOM-04 §3 — Service User Credentials, Platform Token vs Classic API Token decision matrix, state-leakage warning for `dynatrace_api_token`
- AUTOM-95 §4 — `dynatrace_iam_service_user` resource (the IAM-side prerequisite)
- AUTOM-95 §10 — Group bindings (where the Service User gets its IAM policies)
- AUTOM-96 LAB — GitHub Actions worked example using long-lived Platform Tokens (simpler pattern this section upgrades)
- AUTOM-09 §3 + §8 — secrets handling and state-at-rest encryption (orthogonal but related concerns)

> **Sources:** dynatrace_api_token resource (Dynatrace GitHub) — *"This resource requires the API token scopes"* `apiTokens.read` and `apiTokens.write`: the classic-API-Token exception above.

12. Best Practices

Security

Practice	Description
Masked secrets	Always mask API tokens in CI/CD
Protected branches	Require reviews for main branch
Environment protection	Manual approval for production
Token rotation	Rotate tokens regularly
Least privilege	Minimal token scopes
External Secrets	Use ESO, SOPS, or Sealed Secrets for K8s
Vault for runtime creds	Retrieve tokens at pipeline runtime instead of static CI/CD secrets
Combined auth	Use Platform Token + API Token for full resource coverage (v1.88.0)
Policy-as-code	OPA/Conftest or Sentinel to enforce resource allowlists and mandatory tagging
State file access	Lock down HCP Terraform workspace permissions — state files may contain OAuth credentials
Two-pipeline model	Separate IAM management (central team) from config management (LOB teams)
Single SA writer	Only the pipeline SA writes to production; all humans get read-only access

Workflow Design

Practice	Description
Validate first	`monaco deploy --dry-run` (structure only — no API calls) or `terraform validate` before deploy
Plan PRs	`terraform plan` shows what would change; Monaco's dry run does not
Plan as PR comment	Post `terraform plan` output directly on PRs for reviewer context
Staged rollout	Dev → Staging → Production
Manual gates	Require approval for production
Rollback plan	Know how to revert changes
Health checks	Verify deployment success
Drift detection	Schedule `terraform plan -detailed-exitcode` to catch ClickOps
Reusable workflows	`workflow_call` for consistent deployment across repos

GitOps Tool Selection

Use Case	Recommended Tool
Simple CI/CD	GitHub Actions / GitLab CI
K8s-native, UI preferred	ArgoCD
K8s-native, Helm-first	FluxCD
Multi-cluster enterprise	ArgoCD or FluxCD

Pull Request Template

.github/PULL_REQUEST_TEMPLATE.md:

```
&lt;a id="dynatrace-configuration-change"&gt;&lt;/a&gt;
## Dynatrace Configuration Change
### Description
&lt;!-- What configuration is being changed? --&gt;

### Type of Change
- [ ] New configuration
- [ ] Modification to existing config
- [ ] Deletion of config

### Environments Affected
- [ ] Development
- [ ] Staging
- [ ] Production

### Validation
- [ ] `monaco deploy --dry-run` / `terraform plan` passed
- [ ] Dry run successful
- [ ] Tested in dev environment

### Rollback Plan
&lt;!-- How to revert if issues arise? --&gt;
```

Notification Integration

Add deployment notifications:

```
# GitHub Actions notification step
- name: Notify Slack
  if: always()
  uses: slackapi/slack-github-action@v4
  with:
    webhook: ${{ secrets.SLACK_WEBHOOK }}
    webhook-type: incoming-webhook   # required since v2; omitted, v1 targeted Workflow Builder
    payload: |
      {
        "text": "Dynatrace config deployment: ${{ job.status }}",
        "blocks": [
          {
            "type": "section",
            "text": {
              "type": "mrkdwn",
              "text": "*Deployment to ${{ github.ref_name }}*\nStatus: ${{ job.status }}\nCommit: ${{
github.sha }}"
            }
          }
        ]
      }
```

> **`slack-github-action` v2+ needs `webhook-type`.** The v2.0.0 release notes (Slack GitHub) say *"This Action now requires setting the webhook-type as a step input value."* — and that before v2 *"The webhook type could be omitted to send to Workflow Builder."* The webhook URL moves from the `SLACK_WEBHOOK_URL` environment variable to the `webhook` input. v4 parses multi-line YAML values more strictly, so keep the payload indented under `payload: |`.

---

13. Governance Architecture

The previous sections covered individual governance tools (Vault, OPA, Sentinel, drift detection). This section synthesizes them into a cohesive **enterprise governance architecture** for Dynatrace configuration management.

The Two-Pipeline Model

Enterprise Dynatrace governance requires **two distinct pipelines** with separate identities, scopes, and cadences:

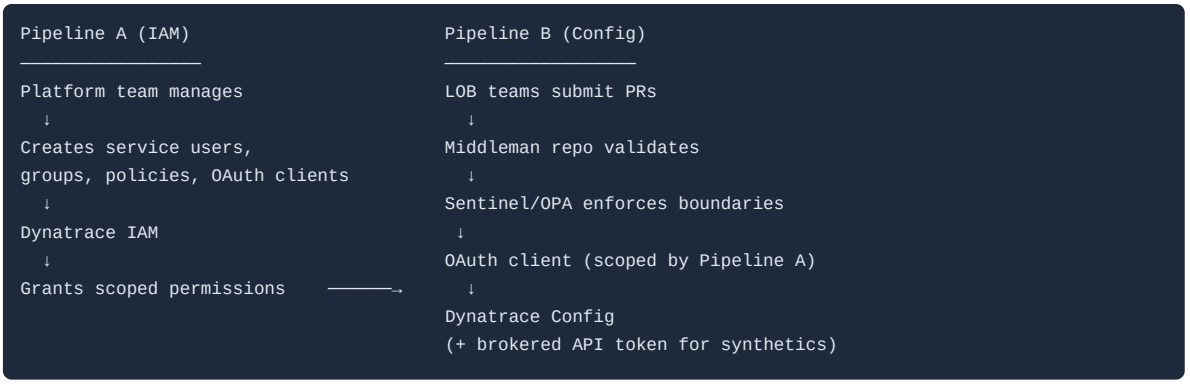
Pipeline A — IAM Pipeline ("Who can do what")

Aspect	Detail
Owner	Central Platform / Observability team
Identity	Service User + OAuth client scoped to IAM resources
Manages	Groups, policies, policy bindings, environment-level permissions (`dynatrace_iam_*`)
Governance	Sentinel/OPA prevents privilege escalation (no `account:admin` , no cross-team bindings)
Cadence	Infrequent — runs when teams onboard, roles change, or environments are provisioned
Repo	`dt-global-config` or equivalent central IAM repo

Pipeline B — Configuration Pipeline ("The actual work")

Aspect	Detail
Owner	Per-team / per-LOB (via middleman repo or brokered self-service)
Identity	Service User + OAuth client scoped to the config domains Pipeline A granted
Manages	Workflows, dashboards, segments, SLOs, anomaly detectors — plus classic alerting profiles, auto-tags and management zones only while the tenant is not yet upgraded
Auth	Dual-auth (OAuth + API token) when synthetics are in scope
Governance	Sentinel/OPA enforces naming, ownership tagging, allowed resource types (and MZ boundaries on classic tenants)
Cadence	Frequent — runs on every config change promotion through environments
Repo	`dt-lob-<lobname>` or team-specific config repos

Pipeline B **can only do what Pipeline A has granted**. Its OAuth client's effective permissions are bounded by the IAM policies, group memberships, and environment scoping that Pipeline A defined.

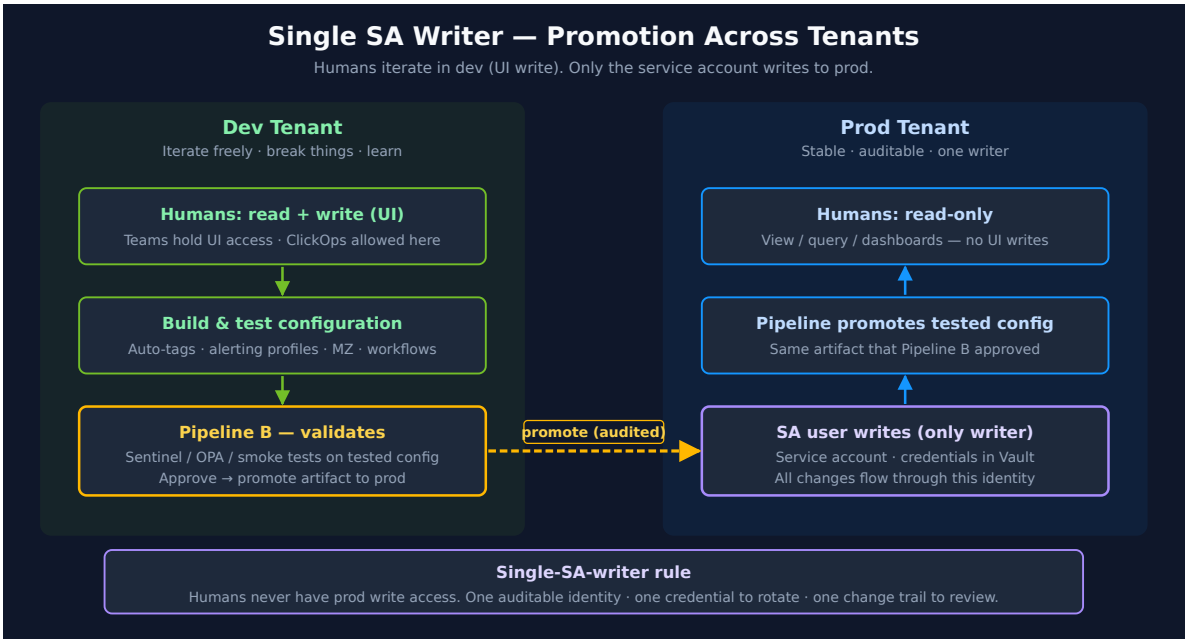


> **Key principle:** Pipeline A creates the identities and permissions. Pipeline B operates under those grants. Separation of concerns between access governance and configuration management.

Operational Model: Single SA Writer

In practice, the two-pipeline architecture produces a clear operational rule: **all human users have read-only access in production, and only one service account can write.**

Teams build and validate configuration in a **dev/test tenant** where they have UI write access. Once tested, the provisioning pipeline — running as the single SA (service account) — promotes that configuration to production. No human ClickOps in prod, and a single auditable write path.



This model works with or without Sentinel. It directly addresses the access scoping challenge: within a single tenant, you cannot easily separate write access between teams for resources that need a classic API token — such as the Terraform provider's synthetic monitors. By splitting dev (where humans create) from prod (where only the pipeline writes), you get isolation through environment boundaries rather than token scoping. The SA's credentials live in Vault or HCP Terraform — never in human hands.

Defense-in-Depth Governance Layers

No single tool provides complete governance. Enterprise Dynatrace configuration management uses **layered controls** where each layer compensates for the limitations of the others:

Layer	Role	Tool / Mechanism
Dynatrace IAM	Source of truth for access control	IAM policies, groups, bindings (see **AUTOM-04**)
Service User + OAuth	Scoped pipeline identity for Gen3 resources	OAuth client credentials
Dual Auth (OAuth + API)	Full coverage when pipeline manages both Gen3 and v1 resources	Combined provider config
Sentinel / OPA	Governance guardrails on what Terraform can do	Policy-as-code (see above)
Vault	Runtime credential retrieval and read auditing (expiry comes from the token itself or §11)	HashiCorp Vault (see above)
PR Review + CODEOWNERS	Human approval gate	GitHub/GitLab branch protection
Module Allowlists	Only approved Terraform modules in use	Sentinel module restrictions or repo-level controls
State File Access Control	Prevent credential extraction from Terraform state	HCP Terraform workspace permissions

> **State file risk:** If a team can read the state of a middleman workspace in HCP Terraform, they could potentially extract the OAuth client credentials or see IAM bindings they should not access. Lock down workspace-level permissions in

> HCP Terraform appropriately.

What Sentinel / OPA Can and Cannot Do

Sentinel / OPA CAN	Sentinel / OPA CANNOT
Restrict which Terraform modules and resource types may be used	Integrate with Dynatrace IAM at runtime
Enforce naming conventions, ownership tagging, and team boundaries	Reduce the runtime permissions of a Dynatrace token
Prevent privilege escalation in IAM-as-code	Replace Dynatrace RBAC / IAM policies
Validate team-to-object ownership in brokered self-service	Run outside HCP Terraform / Terraform Enterprise (Sentinel only — use OPA/Conftest for CI)

> Sentinel does not *grant* access. Sentinel decides whether Terraform is *allowed to grant* access. Dynatrace IAM remains the source of truth for what the pipeline identity can actually do.

Framing for Security Reviewers

When presenting this architecture to security teams or auditors, document the classic-API-token limitation for synthetics as an **accepted constraint of the Terraform path, with compensating controls**. Be precise about its scope: the provider's synthetic resources require a classic API token (`dynatrace_http_monitor` : *"This resource requires the API token scope"* `ExternalSyntheticIntegration`), and a classic token cannot be scoped per team. Dynatrace IAM, however, now lists `synthetic:monitors:write` with a `synthetic:dt.security_context` condition — *"Restricts the permission to synthetic monitors assigned to the matching security context."* — so check whether that scoping covers the API path your pipeline uses before calling the gap a platform limit.

> **Sources:** dynatrace_http_monitor (Dynatrace GitHub), IAM policy statements reference (DT docs).

The compensating controls:

1. **Gen3/platform resources** — genuinely scoped via Service User + OAuth + IAM policies
2. **Synthetic monitors via Terraform** — brokered API token access through central pipeline (teams never hold the token directly)
3. **Pipeline guardrails** — Sentinel/OPA enforce naming, ownership tagging, team boundaries, and resource type restrictions
4. **Credential management** — Vault provides runtime retrieval and access audit logging; per-job token expiry comes from the §11 minting pattern
5. **Audit trail** — Git history + PR reviews + Dynatrace audit logs provide full change traceability

This framing is much stronger than trying to pretend the classic-token scoping gap does not exist. It demonstrates a mature, defense-in-depth approach to a known platform limitation.

---

14. Next Steps

Deployment Event Tracking

Send deployment events to Dynatrace:

```
- name: Send Deployment Event
  run: |
    curl -X POST "${{ secrets.DT_URL }}/api/v2/events/ingest" \
      -H "Authorization: Api-Token ${{ secrets.DT_TOKEN }}" \
      -H "Content-Type: application/json" \
      -d '{
        "eventType": "CUSTOM_DEPLOYMENT",
        "title": "Dynatrace Config Deployment",
        "properties": {
          "source": "github-actions",
          "commit": "${{ github.sha }}",
          "branch": "${{ github.ref_name }}"
        }
      }'
```

Scope the event to what was actually deployed

The call above deliberately carries **no `entitySelector`**, and for this particular job that is defensible: it marks a *Dynatrace configuration* deployment, which has no application entity behind it. `entitySelector` is optional, and when it is omitted "the event is associated with the environment (`dt.entity.environment`) entity" — environment scope, which is the honest scope for a tenant-wide config change.

The trap is copying this shape for an application release, which is what most teams reach for next. An environment-scoped event cannot appear in any entity-scoped view, so a release marker sent this way never lands on the timeline of the service it released, and nothing downstream can filter it to that service. Attribute those:

```
- name: Send Deployment Event
  run: |
    curl -X POST "${{ secrets.DT_URL }}/api/v2/events/ingest" \
      -H "Authorization: Api-Token ${{ secrets.DT_TOKEN }}" \
      -H "Content-Type: application/json" \
      -d '{
        "eventType": "CUSTOM_DEPLOYMENT",
        "title": "Deploy checkout-service ${{ github.ref_name }}",
        "entitySelector": "type(\"SERVICE\"),tag(\"app:checkout\")",
        "properties": {
          "source": "github-actions",
          "commit": "${{ github.sha }}",
          "branch": "${{ github.ref_name }}"
        }
      }'
```

Two caveats worth knowing before you build on this. Entity selectors only match entities **active within the last 24 hours** — the `entityId` filter is the documented exception for inactive entities — so a selector for a service that has just been created may match nothing on the very first deploy. And `CUSTOM_DEPLOYMENT` is an annotation event type: it does not open a problem, so the failure mode here is a marker that silently goes to the wrong place, not alert noise. The alerting version of this mistake — an unattributed event type that *does* open problems, where the environment fallback instead merges unrelated alerts into one unactionable problem — is covered in S2D-05 and AIOPS-02 §4.

For application release identity, prefer the environment-variable path over an API call. Setting `DT_RELEASE_VERSION`, `DT_RELEASE_PRODUCT`, `DT_RELEASE_STAGE`, and `DT_RELEASE_BUILD_VERSION` on the deployed process lets OneAgent attach

release identity at source, so every signal from that process carries it without a pipeline step that can fail silently. FAQ-02 §3 covers these under the four-source hierarchy.

> **Sources:** Ingest an event — POST /api/v2/events/ingest (DT docs) — `entitySelector` optional, environment fallback, and the 24-hour-active constraint with its `entityId` exception. **Derived:** "an environment-scoped event cannot appear in an entity-scoped view" follows from the association rule plus entity-scoped filtering; it is not stated as such in the API reference.

Latest Dynatrace: SDLC deployment events

On the latest Dynatrace, CI/CD deployment activity has a dedicated ingest endpoint, and it is what the pipeline-observability samples in the next section use. The Events API v2 call above remains valid; this is the path the Release Monitoring dashboard reads:

```
- name: Send SDLC deployment event
  run: |
    curl -X POST "${{ secrets.DT_URL }}/platform/ingest/v1/events.sdlc" \
      -H "Authorization: Bearer ${{ secrets.DT_PLATFORM_TOKEN }}" \
      -H "Content-Type: application/json" \
      -d '[{
        "event.type": "deployment",
        "event.status": "finished",
        "event.category": "task",
        "event.provider": "github-actions",
        "cicd.deployment.name": "deploy checkout-service",
        "cicd.deployment.release_stage": "production",
        "cicd.deployment.status": "succeeded",
        "task.outcome": "Success"
      }]'
```

Here `DT_URL` is the environment URL (`https://{environment-id}.live.dynatrace.com`) and the platform token needs `openpipeline:events.sdlc:ingest` . The docs describe the deployment-finished event as *"the event type the Release Monitoring dashboard consumes"*; a `202 Accepted` response means it was received.

> **Sources:** Version detection strategies (DT docs) — *"This is the event type the Release Monitoring dashboard consumes."* SDLC events (DT docs) — endpoint `/platform/ingest/v1/events.sdlc` , required scope `openpipeline:events.sdlc:ingest` .

Continue the Series

Next Notebook	Focus
AUTOM-08: Migration Automation	Bulk configuration transfer between tenants

Additional Resources

- GitHub Actions Documentation
- GitLab CI/CD Documentation
- ArgoCD Documentation
- FluxCD Documentation
- Dynatrace Operator
- Monaco GitHub Repository
- External Secrets Operator
- OPA/Conftest

- Sentinel Documentation
- HashiCorp Vault Action
- Dynatrace OAuth Client Guide

Summary

In this notebook, you learned:

- GitOps fundamentals for Dynatrace configuration
- Setting up GitHub Actions and GitLab CI/CD pipelines
- **Combined auth** (Platform Token + API Token) for full resource coverage in Terraform pipelines
- **Vault integration** for runtime credential retrieval and read auditing (per-job token expiry comes from the §11 minting pattern)
- **Policy-as-code gates** using OPA/Conftest and Sentinel for governance enforcement
- **Sentinel limitations** — it governs Terraform plans, not Dynatrace API permissions at runtime
- **Drift detection** workflows to catch manual UI changes
- **Reusable workflows** (`workflow_call`) for multi-team organizations
- ArgoCD integration with External Secrets for token management
- FluxCD with HelmRelease and SOPS for secret encryption
- Dynatrace Operator GitOps patterns for multi-cluster and multi-tenant environments (one OneAgent DynaKube per node pool)
- **Two-pipeline model**: Pipeline A (IAM) creates identities, Pipeline B (Config) applies configuration under those grants
- **Single SA writer**: only the pipeline SA writes to production; humans develop in dev/test
- **Defense-in-depth**: layered governance from Dynatrace IAM through Sentinel/OPA, Vault, PR gates, and state file controls

> **Key Takeaway:** CI/CD integration transforms Dynatrace config management into a software development workflow. Use a two-pipeline model to separate IAM governance from configuration management. Enforce single-SA-writer in production. Layer Vault, OPA/Sentinel, and Dynatrace IAM policies for defense-in-depth — no single tool provides complete governance.

---

*Continue to **AUTOM-08: Migration Automation** to learn bulk configuration transfer.*

> **Site Reliability Guardian, in depth:** the CLI and repos below wrap SRG automation for CI/CD, but this notebook does not otherwise cover guardian mechanics. For the objective/threshold model, the result classification, creating and triggering a guardian, and the Terraform/Monaco config-as-code schema, see **SLO-06: Site Reliability Guardians**.

Community Resources & CI/CD Examples

The following GitHub repositories provide working CI/CD pipelines, validation scripts, and platform engineering references:

Pipeline Observability Samples (in `community-examples/configuration-as-code` — moved from the archived `dynatrace-configuration-as-code-samples` repo)

Both Monaco and Terraform variants are available for each CI/CD platform:

CI/CD Platform	Monaco Sample	Terraform Sample
GitHub Actions	`github_pipeline_observability`	`github_pipeline_observability_terraform`
GitLab CI	`gitlab_pipeline_observability`	`gitlab_pipeline_observability_terraform`
Azure DevOps	`azure_devops_observability`	--
ArgoCD	`argocd_observability`	`argocd_observability_terraform`

All samples ingest SDLC events via OpenPipeline and include pre-built dashboards.

CI Validation Scripts (in `configuration-as-code/scripts/ci/`)

Script	Purpose
`validate-monaco.sh`	Validate Monaco configuration in CI
`validate-terraform.sh`	Validate Terraform configuration in CI
`check-secrets.sh`	Detect hardcoded secrets before commit

CI/CD Automation Tools

Repository	Description
dtctl (source: dynatrace-oss/dtctl)	The Dynatrace command-line tool, for Latest Dynatrace on SaaS only — *"dtctl is compatible with Dynatrace SaaS environments only."* In a pipeline job, use a platform token (*"In cases where OAuth isn't possible, such as in headless environments, you can also use platform tokens."*) passed as `DTCTL_ENVIRONMENT` / `DTCTL_TOKEN`. Preview a declarative change with `dtctl apply -f <file> --dry-run`, and add `--check-scopes` to a command to see the scopes it requires without running it.
dynatrace-automation-tools	**Archived** — the README states *"This repository has been archived and is no longer maintained."* and points to dtctl for similar use cases. Previously: CLI for Site Reliability Guardian automation and deployment event ingestion in CI/CD

Platform Engineering References

Repository	Description
platform-engineering-demo	Full IDP reference: ArgoCD + Backstage + Keptn + OpenTelemetry + Dynatrace
demo-crossplane	Crossplane + Terraform provider for continuous GitOps reconciliation (no manual pipeline triggers)
obslab-composite-srg	Composite Site Reliability Guardians with GitHub Actions
obslab-release-validation	Release validation using k6, business events, workflows, and SRG

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.